



การจำลองการเคลื่อนที่แบบบราวน์สองมิติและทฤษฎีบทความผันผวนในคิมจับเชิงแสง

ปฐมพร เนียมรักษา

ปริญญาานิพนธ์เสนอต่อมหาวิทยาลัยราชภัฏรำไพภิรมย์
เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญา
วิทยาศาสตรบัณฑิต (ฟิสิกส์)
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยราชภัฏรำไพภิรมย์

การจำลองการเคลื่อนที่แบบบราวน์สองมิติและทฤษฎีบทความผันผวนในคิมจับเชิงแสง

ปฐมพร เนียมรักษา

ปริญญานิพนธ์เสนอต่อมหาวิทยาลัยรามคำแหง
เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญา
วิทยาศาสตร์บัณฑิต (ฟิสิกส์)
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยรามคำแหง

SIMULATION OF 2D BROWNIAN MOTION AND THE FLUCTUATION THEOREM IN
OPTICAL TWEEZERS

PATHOMPORN NIAMRAKSA

A THESIS PRESENTED TO RAMKHAMHAENG UNIVERSITY
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE DEGREE OF BACHELOR OF SCIENCE
(PHYSICS)

2025

COPYRIGHTED BY RAMKHAMHAENG UNIVERSITY

บทคัดย่อ

ชื่อเรื่อง : การจำลองการเคลื่อนที่แบบบราวน์สองมิติ
และทฤษฎีบทความผันผวนในคิมจับเชิงแสง
ผู้เขียน : นาย ปฐมพร เนียมรักษา
ชื่อปริญญา : วิทยาศาสตร์บัณฑิต
สาขาวิชา : ฟิสิกส์
ปีการศึกษา : 2568
อาจารย์ที่ปรึกษา : ผศ.ดร. มาณิต แก้วทนต์

การเคลื่อนที่แบบบราวน์เป็นการเคลื่อนที่แบบสุ่มของอนุภาคที่อยู่ในของเหลว ต่อมา ได้มีการอธิบายเพิ่มเติมโดยการเสนอแนวคิดกระบวนการแพร่และสร้างสมการของการเคลื่อนที่แบบบราวน์ให้อยู่ในรูปของกฎการเคลื่อนที่ของนิวตัน ที่รู้จักในชื่อสมการล็องจีแวง คิมจับเชิงแสงเป็นการดักจับอนุภาคด้วยเลเซอร์ลำแสงเกาส์เซียน TEM_{00} โดยอาศัยสองแรงหลักคือ แรงเกรเดียนต์ และแรงกระเจิง ทั้งสองแรงนี้ทำให้เกิดการดักจับอนุภาคขึ้น ทฤษฎีบทความผันผวนอธิบายว่า สำหรับระบบจำกัดและเวลาจำกัด ความน่าจะเป็นที่การไหลของฟลักซ์สูญสลายจะย้อนทิศทางเทียบกับทิศทางปกติที่สอดคล้องกับกฎข้อที่สองของอุณหพลศาสตร์มีค่าเท่าใด โครงงานนี้มีวัตถุประสงค์เพื่อศึกษาการเคลื่อนที่แบบบราวน์ในกระบวนการต่าง ๆ เช่น กระบวนการแพร่ในของเหลว และกระบวนการดักจับเชิงแสงด้วยการจำลองจากสมการล็องจีแวง ผลการจำลองที่ได้แสดงให้เห็นว่าฟังก์ชันการสลายตัวและทฤษฎีบทความผันผวนชั่วคราวสอดคล้องกับทฤษฎีที่ทำนายไว้

ABSTRACT

Title : Simulation of 2D Brownian Motion
and The Fluctuation Theorem in Optical Tweezers
Author : Mr. Pathomporn Niamraksa
Degree : Bachelor of Science
Major : Physics
Academic Year : 2025
Advisor : Asst.Prof.Dr. Manit Klawtanong

Brownian motion is the stochastic movement of a particle in a fluid. Later, it was further explained by introducing the concept of diffusion, leading to an equation for Brownian motion in the form of Newton's laws of motion, known as the Langevin equation. Optical tweezers trap particles using a Gaussian laser beam in the fundamental mode TEM_{00} . The trapping mechanism relies on two primary forces: the gradient force and the scattering force. The fluctuation theorem gives an analytic expression for the probability that, for a finite system and for a finite time, the dissipative flux flows in the reverse direction to that required by the second law of thermodynamics. This project aims to study Brownian motion in different processes, such as a diffusion process in a viscous fluid and an optical trapping process modeled via the Langevin equation. The results show that the dissipation function and integrated fluctuation theorem are consistent with the theoretical prediction.

กิตติกรรมประกาศ

โครงการฉบับนี้สำเร็จลุล่วงได้ด้วยความกรุณาอย่างยิ่งจาก ผู้ช่วยศาสตราจารย์ ดร. มา-
ณิต แก้วทนต์ ที่ช่วยดูแลให้คำปรึกษาจนโครงการนี้สำเร็จลุล่วงได้ด้วยดี ผู้จัดทำโครงการ
ขอขอบพระคุณเป็นอย่างสูง

สุดท้ายขอขอบพระคุณ บิดา มารดา ครูบาอาจารย์ ที่คอยสั่งสอน ให้กำลังใจ และ ให้
คำปรึกษาตลอดเสมอมา

ปฐมพร เนียมรักษา

สารบัญ

	หน้า
บทคัดย่อภาษาไทย	(1)
บทคัดย่อภาษาอังกฤษ	(2)
กิตติกรรมประกาศ	(3)
สารบัญรูป	(6)
คำอธิบายสัญลักษณ์และคำย่อ	(7)
บทที่	
1 บทนำ	1
1.1 ความเป็นมาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์	1
1.3 ขอบเขตของโครงการ	1
1.4 ประโยชน์ที่คาดว่าจะได้รับ	2
2 วรรณกรรมที่เกี่ยวข้อง	3
2.1 กฎของสโตกส์ (Stokes's Law)	3
2.2 การเคลื่อนที่แบบบราวน์ (Brownian Motion)	5
2.2.1 นิยามของการเคลื่อนที่แบบบราวน์	7
2.3 สมการล็องจีแวง (Langevin Equation)	8
2.3.1 พลศาสตร์ของล็องจีแวง (Langevin Dynamics)	9
2.3.2 สมการการแพร่ (Diffusion Equation)	11
2.3.3 สมการล็องจีแวงทั่วไป (Generalized Langevin Equation)	12
2.4 คีมจับเชิงแสง (Optical Tweezers)	13
2.5 ทฤษฎีบทความผันผวน (The Fluctuation Theorem)	15
2.5.1 ฟังก์ชันการสูญเสีย (The Dissipation Function)	17
2.5.2 ฟังก์ชันการสูญเสียสำหรับสมการล็องจีแวง (Langevin Derivation of Dissipation Function)	21
2.5.3 ทฤษฎีบทความผันผวนชั่วครู่ (Transient Fluctuation Theorem)	23
2.5.4 ฟังก์ชันการสูญเสียสำหรับการทดสอบการจับอนุภาค (The Dissipation Function for Capture Experiment)	24
3 วิธีดำเนินการวิจัย	27
3.1 การเคลื่อนที่แบบบราวน์ 2 มิติ	27
3.2 คีมจับเชิงแสง	28

บทที่	หน้า
3.3 ฟังก์ชันการสูญเสีย	29
3.4 รูปแบบการดำเนินการ	30
3.5 เครื่องมือในการจำลอง	30
3.6 ขั้นตอนในการจำลอง	30
3.7 การวิเคราะห์ข้อมูล	31
3.8 การเปรียบเทียบผลของข้อมูล	32
4 ผลและการอภิปรายผล	33
4.1 การเคลื่อนที่แบบบราวน์ 2 มิติ	33
4.1.1 สมการกระบวนการแพร่	34
4.1.2 คีมจับเชิงแสง	36
4.2 ทฤษฎีบทความผันผวน	38
4.2.1 ฟังก์ชันการสูญเสีย	38
4.2.2 ทฤษฎีบทความผันผวนชั่วคราว	43
5 สรุปผลและข้อเสนอแนะ	45
5.1 สรุปผล	45
5.2 ข้อเสนอแนะ	46
บรรณานุกรม	47
ภาคผนวก	53
ก ซอร์สโค้ด (Source Code)	54
ก.1 ซอร์สโค้ดภายนอก	55
ก.2 การเคลื่อนที่แบบบราวน์ 2 มิติ	56
ก.2.1 จินตทัศน์	56
ก.2.2 สมการกระบวนการแพร่	66
ก.2.3 คีมจับเชิงแสง	68
ก.3 ทฤษฎีบทความผันผวน	70
ประวัติผู้เขียน	77

สารบัญรูป

รูปที่		หน้า
2.1	การไหลแบบคงที่ของของไหลที่เลขเรย์โนลด์ส์ต่ำ ๆ ผ่านทรงกลม	3
2.2	การกระจายความดัน p ของของไหลที่เลขเรย์โนลด์ส์ต่ำ ๆ บนพื้นผิวทรงกลมรัศมี R	4
2.3	การกระจายความเค้นเฉือน τ_o ของของไหลที่เลขเรย์โนลด์ส์ต่ำ ๆ บนพื้นผิวทรงกลมรัศมี R	4
2.4	ต้นคลาร์เคียพูลเซลลา	5
2.5	ละอองเกสรของดอกคลาร์เคียพูลเซลลา	6
2.6	การเคลื่อนที่แบบสุ่ม	6
2.7	กระบวนการแพร่ในน้ำ	7
2.8	การเคลื่อนที่แบบบราวน์ในระบบ 1 มิติ	8
2.9	ทิศทางของแรงที่กระทำกับอนุภาค	9
2.10	หลักการของคิมจับเชิงแสง	14
2.11	ภาพประกอบของเส้นคอนทัวร์ของพลังงานศักย์	14
2.12	การแจกแจงความน่าจะเป็นของ $P_{xy\tau}$ ของเทนเซอร์ความดัน	16
2.13	อัตราส่วนความน่าจะเป็นเชิงลอการิทึม $\Pi(P_{xy\tau})$	17
2.14	ภาพประกอบของเซตของเส้นทางที่ใกล้เคียงกัน	20
2.15	ภาพประกอบของเซตเส้นทางสุ่ม	22
3.1	ทิศทางแรงของสมการลี้จิงวิ่งด้วยคิมจับเชิงแสง	29
4.1	ภาพประกอบจินตทัศน์ของเส้นทางในการเคลื่อนที่แบบบราวน์ 2 มิติ	33
4.2	ภาพประกอบจินตทัศน์ของเส้นทางในการเคลื่อนที่แบบบราวน์ 2 มิติ ด้วยคิมจับเชิงแสง	34
4.3	เส้นทางของการเคลื่อนที่แบบบราวน์ 2 มิติ ในน้ำ	35
4.4	เส้นทางของการเคลื่อนที่แบบบราวน์ 2 มิติ ในของเหลวประเภทต่าง ๆ	36
4.5	เส้นทางของการเคลื่อนที่แบบบราวน์ 2 มิติ ด้วยคิมจับเชิงแสง	37
4.6	ภาพขยายของเส้นทาง $(1.22, 1.22) \times 10^{-6}$ N/m	37
4.7	เส้นทางของการเคลื่อนที่แบบบราวน์ 2 มิติ ด้วยคิมจับเชิงแสงที่รัศมีต่าง ๆ	38
4.8	ฟังก์ชันการสูญเสีย	40
4.9	ความน่าจะเป็นของฟังก์ชันการสูญเสีย	41
4.10	ค่าเฉลี่ยของคอมเบลของฟังก์ชันการสูญเสียเทียบกับเวลา $t(s)$	42
4.11	ทฤษฎีบทความผันผวนชั่วครู่	43
4.12	ลอการิทึมของทฤษฎีบทความผันผวนชั่วครู่	44

คำอธิบายสัญลักษณ์และคำย่อ

Re	คือ เลขเรย์โนลด์ส (Reynolds number)
R	คือ รัศมีของอนุภาค (Particle radius)
k_B	คือ ค่าคงที่บ็อลทซ์มันน์ (Boltzmann constant)
T	คือ อุณหภูมิสัมบูรณ์ (Absolute temperature)
η	คือ ความหนืดพลวัต (Dynamic viscosity)
D	คือ สัมประสิทธิ์การแพร่ (Diffusion coefficient)
TEM_{00}	คือ Transverse electromagnetic mode 00
FPS	คือ เฟรมต่อวินาที (Frames per second)
$\mathcal{N}(\mu, \sigma^2)$	คือ การแจกแจงปกติ (Normal distribution)
W_t	คือ กระบวนการวีเนอร์ (Wiener process)
γ	คือ สัมประสิทธิ์แรงเสียดทาน (Friction coefficient)
μ	คือ ค่าคงที่การเคลื่อนที่ (Mobility)
$\xi(t)$	คือ สัญญาณรบกวนแบบเกาส์สีขาว (Gaussian white noise)
k	คือ ความแข็งตึงของกับดัก (Trap stiffness)
Ω_t	คือ ฟังก์ชันการสูญเสีย (Dissipation function)
FT	คือ ทฤษฎีบทความผันผวน (Fluctuation theorem)
TFT	คือ ทฤษฎีความผันผวนชั่วคราว (Transient fluctuation theorem)
LHS	คือ ทางด้านซ้าย (Left-hand side)
RHS	คือ ทางด้านขวา (Right-hand side)

บทที่ 1

บทนำ

1.1. ความเป็นมาและความสำคัญของปัญหา

การเคลื่อนที่แบบบราวน์นั้นเป็นการเคลื่อนที่แบบสุ่มของอนุภาคที่อยู่ในของเหลว เพื่อจะเข้าใจกระบวนการแพร่ของอนุภาคให้มากขึ้น เราจำเป็นต้องศึกษาพฤติกรรมของการเคลื่อนที่แบบบราวน์ในของเหลวต่าง ๆ รวมถึงการศึกษาการเคลื่อนที่ของอนุภาคในการดักจับเชิงแสง

ในการทดลอง การทดสอบทฤษฎีบทความผันผวนในสภาวะนอกสมดุลเป็นสิ่งที่ทำได้ยาก เพราะเราต้องติดตามเส้นทางการเคลื่อนที่ของอนุภาคเป็นจำนวนมากเส้นทาง ในโครงการนี้ เราจึงสร้างแบบจำลองและเขียนโปรแกรมคอมพิวเตอร์เพื่อจำลองเส้นทางการเคลื่อนที่แบบบราวน์ในระบบดักจับเชิงแสง เราศึกษาฟังก์ชันการสลายตัวในระบบนอกสมดุล และเปรียบเทียบผลการจำลองทางคอมพิวเตอร์ที่ได้กับทฤษฎีบทความผันผวน

1.2. วัตถุประสงค์

1. เพื่อศึกษาการเคลื่อนที่แบบบราวน์จากสมการลี้จางในรูปร่างต่าง ๆ เช่น สมการกระบวนการแพร่ในของเหลวต่าง ๆ และการดักจับเชิงแสง
2. เพื่อศึกษาฟังก์ชันการสลายตัวและทฤษฎีบทความผันผวน

1.3. ขอบเขตของโครงการ

1. ศึกษาการเคลื่อนที่แบบบราวน์ด้วยสมการการแพร่และจำลองการดักจับเชิงแสงโดยใช้สมการลี้จาง

2. ศึกษาฟังก์ชันการสลายตัวและทฤษฎีบทความผันผวนจากการจำลองการดักจับเชิงแสง

1.4. ประโยชน์ที่คาดว่าจะได้รับ

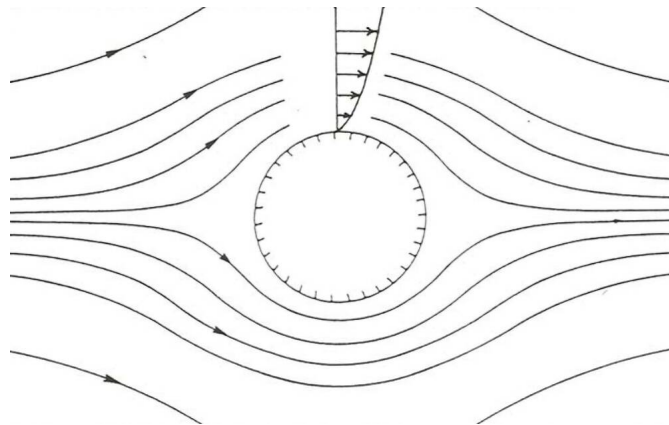
1. เข้าใจการเคลื่อนที่แบบบราวน์ในระบบการดักจับเชิงแสง
2. สร้างแบบจำลองทางคอมพิวเตอร์โดยใช้โปรแกรมไพทอน
3. บอกความแตกต่างของกระบวนการแพร่ในของเหลวที่ค่าความแข็งตึงของก้ำกั๊กและคาร์ซีมีต่าง ๆ ได้
4. เข้าใจฟังก์ชันการสลายตัวและทฤษฎีบทความผันผวนของระบบนอกสมดุล

บทที่ 2

วรรณกรรมที่เกี่ยวข้อง

2.1. กฎของสโตกส์ (Stokes's Law)

กฎของสโตกส์ (Stokes's law) [1] อธิบายการไหลของของไหลผ่านทรงกลมที่เลขเรย์โนลด์ส์ต่ำ ๆ (low Reynolds number) ($Re \ll 1$) ดังรูป 2.1 โดยจะแบ่งเป็นสองส่วนคือ ความดันกับความเค้นเฉือนของความหนืด (viscous shear stress) [35]



รูปที่ 2.1 : การไหลแบบคงที่ของของไหลที่เลขเรย์โนลด์ส์ต่ำ ๆ ผ่านทรงกลม [35]

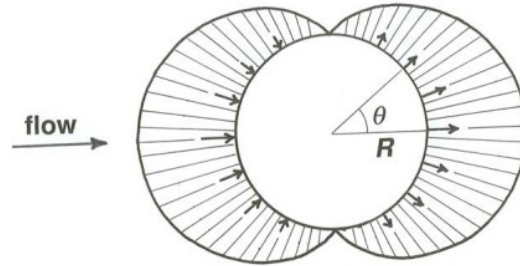
ที่ทุกจุดบนพื้นผิวทรงกลมดังรูป 2.2-2.3 จะมีค่าของความดัน p และความเค้นเฉือน τ_o ของของไหลเป็นไปตามสมการ

$$p = -\frac{3}{2} \frac{\eta U \cos \theta}{R} \quad (2.1)$$

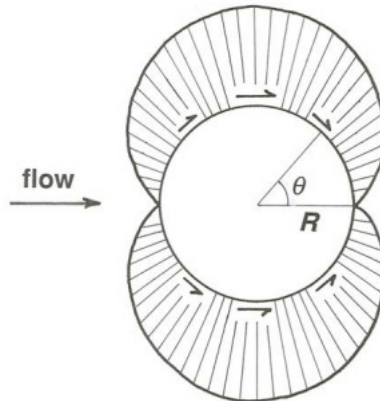
$$\tau_o = \frac{3}{2} \frac{\eta U \sin \theta}{R} \quad (2.2)$$

เมื่อ p คือ ความดันของของไหล

- τ_o คือ ความเค้นเฉือนของของไหล
- η คือ ความหนืดพลวัต (dynamic viscosity) ของของไหล
- U คือ อัตราเร็วของของไหลที่ระยะห่างจากทรงกลมมาก ๆ
- R คือ รัศมีของทรงกลม
- θ คือ ตำแหน่งบนผิวของทรงกลม



รูปที่ 2.2 : การกระจายความดัน p ของของไหลที่เลขเรย์โนลด์ส์ต่ำ ๆ บนพื้นผิวทรงกลม รัศมี R [35]



รูปที่ 2.3 : การกระจายความเค้นเฉือน τ_o ของของไหลที่เลขเรย์โนลด์ส์ต่ำ ๆ บนพื้นผิวทรงกลมรัศมี R [35]

สโตกส์พบว่าแรงต้านการไหลทั้งหมดบนทรงกลมมีค่าเท่ากับ

$$F_{drag} = 6\pi\eta RU \quad (2.3)$$

เมื่อ F_{drag} คือ แรงต้าน (drag force) ที่เลขเรย์โนลด์ส์น้อย ๆ
เนื่องความหนืดพลวัตสามารถเขียนได้ตามสมการ [36]

$$\eta = \rho\nu \quad (2.4)$$

เมื่อ ρ คือ ความหนาแน่นของของไหล (fluid density)
 ν คือ ความหนืดจลนศาสตร์ (kinematic viscosity)

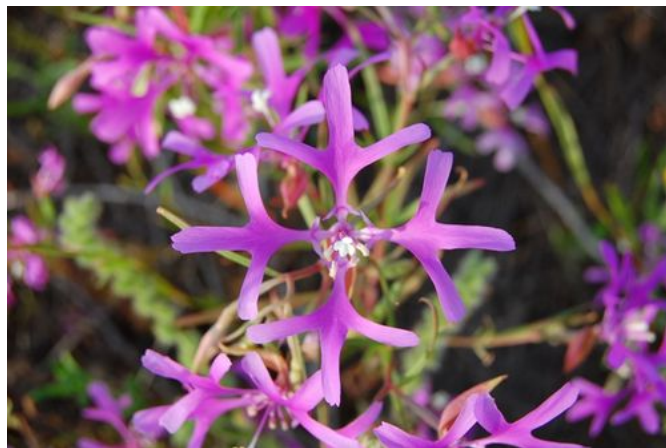
ดังนั้นเมื่อนำสมการ (2.4) มาใส่ในสมการ (2.3) จะทำให้ได้สมการใหม่ คือ

$$F_{drag} = 6\pi\rho\nu RU \quad (2.5)$$

ทำให้ได้สมการ (2.5) เรียกว่ากฎของสโตกส์ [1]

2.2. การเคลื่อนที่แบบบราวน์ (Brownian Motion)

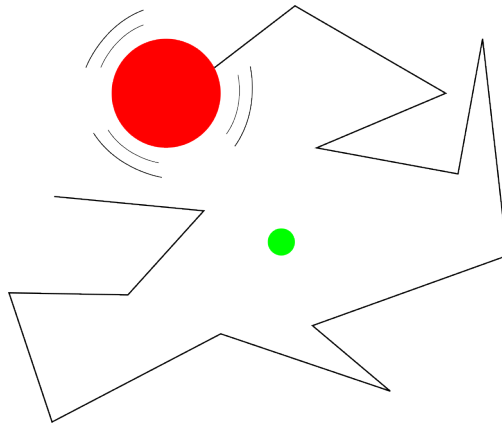
ในปี ค.ศ. 1828 โรเบิร์ต บราวน์ (Robert Brown) [2] ได้ทำการทดลองเกี่ยวกับละอองเรณู (pollen) ของต้นคลาร์เคียพุลเซลลา (ดูรูป 2.4-2.5) (*Clarkia pulchella*) โดยนำละอองเรณูภายในอับเรณู (anther) ที่โตเต็มที่แต่ยังไม่แตกออกมา นำไปทดลองในน้ำพบว่าละอองเรณูมีการเปลี่ยนแปลงตำแหน่งสัมพัทธ์ที่ไม่ได้เกิดจากกระแสน้ำในของเหลว หรือจากการระเหยอย่างช้า ๆ ของของเหลว แต่เป็นคุณสมบัติของตัวอนุภาคเอง โดยอนุภาคนี้เป็นทรงกลม และเคลื่อนที่แบบสุ่ม (random movement) อย่างรวดเร็วดังรูป 2.6



รูปที่ 2.4 : ต้นคลาร์เคียพุลเซลลา [37]



รูปที่ 2.5 : ละอองเรณูของดอกคาร์เคียพูลเซลลาที่แตกออกมา [3]



รูปที่ 2.6 : การเคลื่อนที่แบบสุ่มคือ การเคลื่อนที่แบบส่ายไปส่ายมารอบจุดศูนย์กลาง เมื่อจุดศูนย์กลางเคลื่อนที่ ละอองเรณูก็จะเคลื่อนที่ตามจุดศูนย์กลางไปด้วย

ในปี ค.ศ. 1905 อัลเบิร์ต ไอน์สไตน์ (Albert Einstein) [4] ได้อธิบายเชิงทฤษฎีเกี่ยวกับการเคลื่อนที่แบบบราวน์ที่ขัดแย้งกับอุณหพลศาสตร์แบบดั้งเดิมที่มองว่า “อนุภาคแขวนลอยไม่สร้างความดันออสโมติก” ซึ่งต่างจากไอน์สไตน์ที่ได้เสนอว่า “อนุภาคแขวนลอยมีพฤติกรรมเหมือนโมเลกุลในสารละลาย เมื่อเจือจางมาก” โดยได้แสดงสมการความดันออสโมติก คือ [5]

$$p_{osmotic} = \frac{RT}{N} \nu \quad (2.6)$$

เมื่อ	$p_{osmotic}$	คือ ความดันออสโมติก (osmotic pressure)
	R	คือ ค่าคงที่ของแก๊สสากล (universal gas constant)
	T	คือ อุณหภูมิสัมบูรณ์ (absolute temperature)

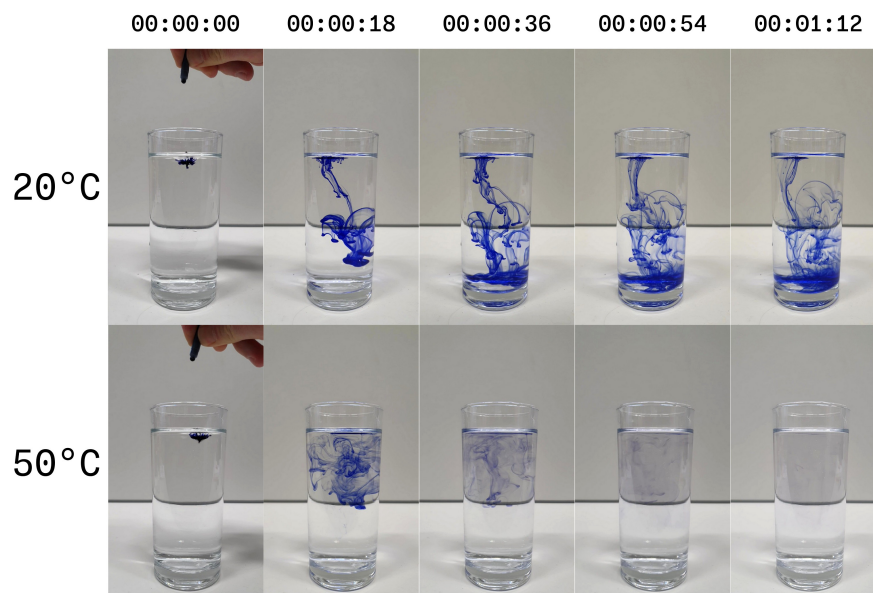
N คือ จำนวนจริงของโมเลกุลใน 1 กรัม-โมเลกุล (gram-molecule)

$\nu = n/V^*$ คือ จำนวนอนุภาคต่อหนึ่งหน่วยปริมาตร

สมการ (2.6) เป็นสมการความดันออสโมติกของอนุภาคแขวนลอย (suspended particles) ซึ่งต่อมาไอน์สไตน์ได้เชื่อมโยงความดันออสโมติกเข้ากับกระบวนการแพร่ (diffusion) ดังรูป 2.7 ทำให้ได้ความสัมพันธ์ของไอน์สไตน์ (Einstein relation)

$$D = \frac{k_B T}{6\pi\eta R} \quad (2.7)$$

เมื่อ D คือ สัมประสิทธิ์การแพร่ (diffusion coefficient)



รูปที่ 2.7 : กระบวนการแพร่ในน้ำ [38]

2.2.1. นิยามของการเคลื่อนที่แบบบราวน์

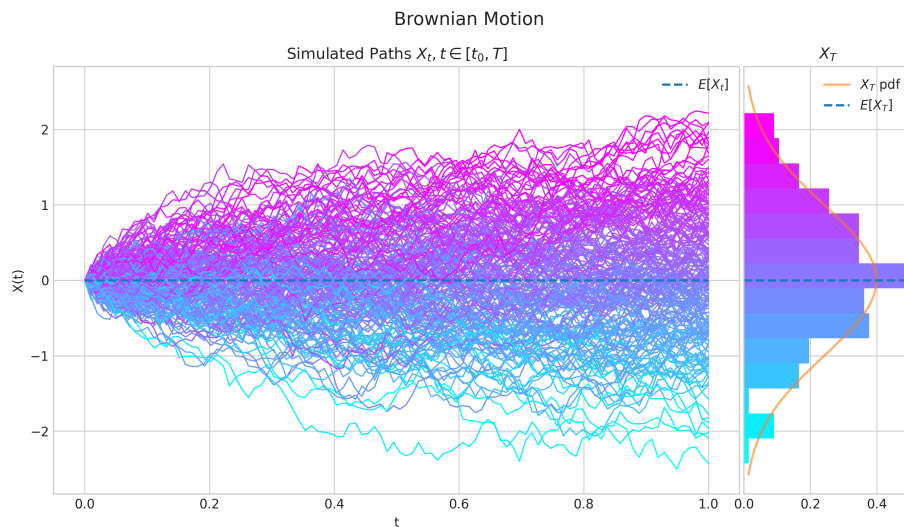
การเคลื่อนที่แบบบราวน์ เรียกอีกอย่างว่า กระบวนการวีเนอร์ (Wiener process) [39] มีความเชื่อมโยงอย่างใกล้ชิดกับการแจกแจงปกติ (normal distribution) แทนด้วย $\mathcal{N}(\mu, \sigma^2)$ [6] โดยตัวแปรสุ่ม X กระจายตัวแบบปกติ (normally distributed) มีค่าเฉลี่ย (mean) μ และความแปรปรวน (variance) σ^2 จะได้ว่าความน่าจะเป็นที่ตัวแปรสุ่ม X ให้ผลลัพธ์มีค่ามากกว่า x ใด ๆ มีเท่ากับ

$$P(X > x) = \frac{1}{\sqrt{2\pi\sigma^2}} \int_x^\infty \exp\left(-\frac{(u-\mu)^2}{2\sigma^2}\right) du, \quad \text{สำหรับทุก } x \in R \quad (2.8)$$

นิยาม กระบวนการสโตแคสติก (stochastic process) $(W_t : t \geq 0)$ เรียกว่า การเคลื่อนที่แบบบราวน์ถ้าเป็นไปตามนี้

- $W_0 = 0$
- W_t เป็นฟังก์ชันต่อเนื่องเกือบจะแน่นอน (almost surely continuous)
- W_t มีการเพิ่มขึ้นเป็นอิสระต่อกัน (independent increments)
- $W_{t+s} - W_s \sim \mathcal{N}(0, t)$ เมื่อ $\mathcal{N}(\mu, \sigma^2)$ แทนการแจกแจงปกติ มีค่าเฉลี่ย μ และความแปรปรวน σ^2

จะได้ว่า $(W_t : t \geq 0)$ เป็นการเคลื่อนที่แบบบราวน์มาตรฐาน (standard Brownian motion) ตัวอย่างการเคลื่อนที่แบบบราวน์แสดงดังรูป 2.8



รูปที่ 2.8 : การเคลื่อนที่แบบบราวน์ในระบบ 1 มิติ [40]

2.3. สมการล็องจีแวง (Langevin Equation)

ปี ค.ศ. 1908 พอล ล็องจีแวง (Paul Langevin) [7] ได้เสนอสมการที่อธิบายการเคลื่อนที่แบบบราวน์ที่อยู่ในรูปของสมการการเคลื่อนที่ของนิวตัน (Newton's laws of motion) “ $F = ma$ ” ที่ประกอบไปด้วย แรงสุ่ม (random force) และแรงลาก (drag force) ที่เป็น

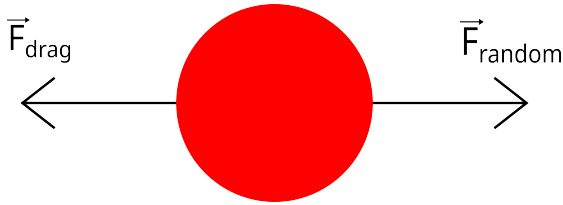
ไปตามกฎของสโตกส์ที่เลขเรย์โนลด์ส์ต่ำ ๆ [8]

2.3.1. พลศาสตร์ของลิ่งจี้แวิ่ง (Langevin Dynamics)

โดยสูตรทั่วไปของสมการลิ่งจี้แวิ่ง [9] สำหรับการเคลื่อนที่อิสระของอนุภาคทรงกลมขนาดใหญ่คือ

$$m \frac{d^2x}{dt^2} = -F_{drag} + F_{random} \quad (2.9)$$

เมื่อ F_{random} คือ แรงสุ่มมีทิศทางดังรูป 2.9



รูปที่ 2.9 : ทิศทางของแรงที่กระทำกับอนุภาค

แรงลาก F_{drag} จะแปรผันตามความเร็วของอนุภาคเป็นไปตามสมการ [10, 11]

$$F_{drag} = \gamma \frac{dx}{dt} \quad (2.10)$$

เมื่อ γ คือ สัมประสิทธิ์แรงเสียดทาน (friction coefficient) มีค่าเป็น $6\pi\eta R$ สำหรับแรงสุ่ม F_{random} สามารถเขียนได้ว่า [10, 11, 12]

$$F_{random} = \xi(t) \quad (2.11)$$

เมื่อ $\xi(t)$ คือ สัญญาณรบกวนแบบเกาส์สีขาว (Gaussian white noise) มีสมบัติดังนี้

$$\langle \xi(t) \rangle = 0 \quad (2.12)$$

$$\langle \xi(t)\xi(t') \rangle = 2\gamma k_B T \delta(t - t') \quad (2.13)$$

เมื่อ $\langle \dots \rangle$ แทนการเฉลี่ยบนตัวแปรสัญญาณรบกวนแบบเกาส์สีขาว สมการ (2.12) ค่าเฉลี่ย

จะหายไป เนื่องจากในเวลาต่าง ๆ จะไม่มีความสัมพันธ์กัน และสมการ (2.13) คือ ฟังก์ชันอัตโนมัติสัมพันธ์ (autocorrelation function) ของ $\xi(t)$

กำหนดให้

$$\frac{dx(t)}{dt} = \dot{x}(t) = v(t) \quad (2.14)$$

สมการลึองจิแวงเขียนใหม่ได้ว่า

$$\begin{aligned} \frac{dv(t)}{dt} &= -\frac{\gamma}{m}v + \frac{1}{m}\xi(t) \\ &= -\frac{1}{\tau}v(t) + \frac{1}{m}\xi(t) \end{aligned} \quad (2.15)$$

เมื่อ $\tau = m/\gamma$ แทนสเกลเวลาของอนุภาค สมการลึองจิแวงเป็นสมการเชิงเส้น คำตอบจะอยู่ในรูปแบบการซ้อนทับกัน (superposition) ของเทอมแรงเสียดทานและ ผลอินทิเกรตที่ขึ้นกับเวลา (time-integrated effect) ของแรงสุ่ม

$$v(t) = \exp\left(-\frac{t}{\tau}\right) \left[v(0) + \frac{1}{m} \int_0^t dt' \exp\left(\frac{t'}{\tau}\right) \xi(t') \right] \quad (2.16)$$

เมื่อเฉลี่ยบนตัวแปรสัญญาณรบกวน และใช้สมบัติ $\langle \xi(t') \rangle = 0$ ตามสมการ (2.12) จะได้ว่า

$$\langle v(t) \rangle = \langle v(0) \rangle \cdot \exp\left(-\frac{t}{\tau}\right) \quad (2.17)$$

มีการเคลื่อนที่แบบแพร่กระจาย (diffusive motion) ดังที่เห็นได้จากความแปรปรวน

$$\langle v(t) v(t') \rangle = \left[\langle v(0)^2 \rangle - \frac{k_B T}{m} \right] \exp\left(-\frac{t+t'}{\tau}\right) + \frac{k_B T}{m} \exp\left(-\frac{|t-t'|}{\tau}\right) \quad (2.18)$$

เมื่อพิจารณา $t = t'$ และที่เวลานานมาก ๆ $t \rightarrow \infty$ ระบบจะอยู่ในสมดุลความร้อน (thermal equilibrium) จะได้ว่า

$$\langle v^2 \rangle_{eq} = \frac{k_B T}{m} \quad (2.19)$$

เมื่อ $\langle \dots \rangle_{eq}$ แทนค่าเฉลี่ยในสมดุล จากทฤษฎีบทการแบ่งเท่ากัน (equipartition theorem) ที่เปรียบเทียบระหว่างพลังงานจลน์ (kinetic energy) กับพลังงานความร้อน (thermal energy)

$$\frac{1}{2}m\langle v^2 \rangle_{eq} = \frac{1}{2}k_B T \quad (2.20)$$

ดังนั้นความผันผวนแบบสุ่ม (random fluctuation) รักษาอนุภาคในการเคลื่อนไหวด้วยความแปรปรวนของความเร็ว

จากสมการ (2.11) และสมการ (2.13) แรงสุ่ม F_{random} สามารถเขียนใหม่ได้ว่า

$$F_{random} = \sqrt{2\gamma k_B T} W(t) \quad (2.21)$$

เมื่อ $W(t)$ คือ กระบวนการวีเนอร์มีการกระจายตัวแบบปกติ $\mathcal{N}(0, 1)$

2.3.2. สมการการแพร่ (Diffusion Equation)

จากสมการ (2.9)-(2.11)

$$\underbrace{m \frac{d^2 x}{dt^2}}_{inertia} = - \underbrace{\gamma \frac{dx}{dt}}_{friction} + \underbrace{\xi(t)}_{white\ noise} \quad (2.22)$$

สมมติให้เทอมแรงเสียดทานใหญ่กว่าเทอมความเฉื่อยมาก ๆ

$$\left| \gamma \frac{dx}{dt} \right| \gg \left| m \frac{d^2 x}{dt^2} \right| \quad (2.23)$$

เนื่องจากเลขเรย์โนลด์ส์มีค่าน้อย ๆ ($Re \ll 1$) [41] และเวลา $t \gg 0$ ทำให้เทอม $m \frac{d^2 x}{dt^2}$ สามารถละทิ้งได้

$$\gamma \frac{dx}{dt} = \xi(t) \quad (2.24)$$

เมื่อหาปริพันธ์จะได้ว่า

$$x(t) = x_0 + \frac{1}{\gamma} \int_0^t dt' \xi(t') \quad (2.25)$$

ไม่มีการเคลื่อนที่ลัพธ์ (motion net) $\langle x(t) - x(0) \rangle = 0$ แต่ว่าความแปรปรวนไม่เป็นศูนย์

$$\langle [x(t) - x(0)]^2 \rangle = \frac{2k_B T}{\gamma} t \propto t \quad (2.26)$$

โดยที่ความแปรปรวนจะเพิ่มขึ้นและแปรผันกับ t สอดคล้องกับกระบวนการแพร่

$$\langle [x(t) - x(0)]^2 \rangle = 2Dt \quad (2.27)$$

เมื่อ D คือ ค่าคงที่การแพร่ (diffusion constant)

$$D = \frac{k_B T}{\gamma} = \mu k_B T = \frac{k_B T}{6\pi\eta R} \quad (2.28)$$

โดยสมการ (2.28) รู้จักกันทั่วไปในชื่อความสัมพันธ์ของไอน์สไตน์ (Einstein relation) [4]
เมื่อ $\mu = 1/\gamma$ คือ ค่าคงที่การเคลื่อนที่ (mobility)

2.3.3. สมการลึงจี้แรงทั่วไป (Generalized Langevin Equation)

สมการลึงจี้แรงทั่วไปคือ สมการทั่วไปของสมการ (2.22) ซึ่งมาจากกฎข้อที่สองของนิวตัน โดยสมมติว่าแรงที่กระทำต่ออนุภาคประกอบด้วย แรงสูก แรงลาก และแรงอนุรักษ์ภายนอกอื่น ๆ [9]

$$m \frac{d^2 x}{dt^2} = m\ddot{x}(t) = F_{drag} + F_{external} + F_{random} \quad (2.29)$$

โดยสัมประสิทธิ์แรงลากถูกพิจารณาว่าเป็นพลวัต ทำให้เขียนแรงลากได้ว่า

$$F_{drag} = - \int_0^t K(t-t') \dot{x}(t') dt' \quad (2.30)$$

เมื่อ $K(t)$ คือ เมมโมรี่เคอร์เนล (memory kernel) ส่วนแรงภายนอกจะพิจารณาเฉพาะแรงอนุรักษ์ (conservative force)

$$F_{external} = - \frac{\partial U(x)}{\partial x} \quad (2.31)$$

เมื่อ $U(x)$ พลังงานศักย์ (potential energy)

ดังนั้น สมการการเคลื่อนที่ของสมการลอสจิงจ์ทั่วไป คือ

$$m\ddot{x}(t) = - \int_0^t K(t-t')\dot{x}(t') dt' - \frac{\partial U(x)}{\partial x} + \sqrt{2\gamma k_B T} W(t) \quad (2.32)$$

สำหรับสมการลอสจิงจ์แบบโอเวอร์แดมป์ (overdamped) หรือลิมิตที่ไม่มีแรงเฉื่อย (inertialless limit) และสัมประสิทธิ์แรงลากเป็นค่าคงที่เขียนได้ว่า

$$\frac{dx}{dt} = \dot{x}(t) = -\frac{1}{\gamma} \frac{\partial U(x)}{\partial x} + \sqrt{\frac{2k_B T}{\gamma}} W(t) \quad (2.33)$$

เราจะใช้สมการนี้ในการศึกษาการกักอนุภาคภายใต้คีมจับเชิงแสง

2.4. คีมจับเชิงแสง (Optical Tweezers)

ในปี ค.ศ. 1970 อาเธอร์ แอชกิน (Arthur Ashkin) [13] ได้ทำการทดลองเกี่ยวกับคีมจับเชิงแสง (optical tweezers) ดังรูป 2.10 โดยทำการทดลองยิงลำแสงเลเซอร์แบบลำแสงเกาส์เซียน (Gaussian beam) TEM_{00} ผ่านเลนส์ทำให้เกิดการแลกเปลี่ยนของโมเมนตัมกับอนุภาค โดยจะมีแรงกระเจิง (scattering force) มีทิศทางไปตามแนวแสง และแรงเกรเดียนต์ (gradient force) มีทิศทางเข้าสู่ศูนย์กลางของแสง จึงทำให้อนุภาคเข้าสู่ศูนย์กลาง เป็นไปตามสมการ [14, 15, 16]

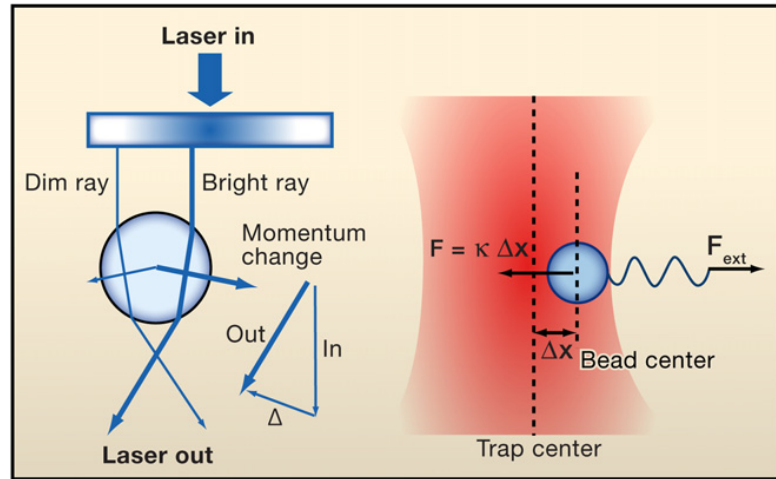
$$F_{trap} = k(x - x_0) = k\Delta x \quad (2.34)$$

ถ้าเรากำหนดให้ตำแหน่งสมดุลเป็นศูนย์ $x_0 = 0$ จะได้

$$F_{trap} = kx \quad (2.35)$$

เมื่อ k คือ ความแข็งตึงของกับดัก (trap stiffness)

x คือ ตำแหน่งของอนุภาค

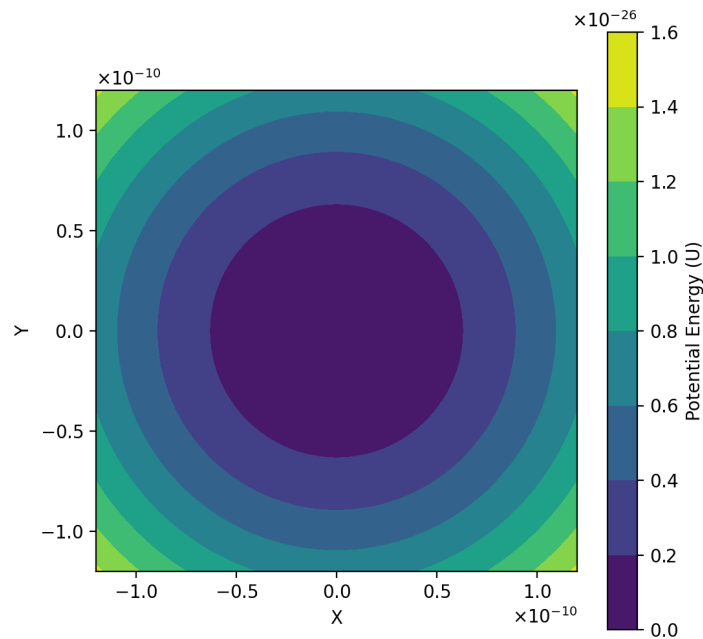


รูปที่ 2.10 : หลักการของคีมจับเชิงแสง [17]

โดยพลังงานศักย์ (potential energy) ของสมการที่ (2.35) คือ [18]

$$U = \int kx \, dx = \frac{1}{2}kx^2 \quad (2.36)$$

รูป 2.11 แสดงตัวอย่างเส้นคอนทัวร์ (contour) ของพลังงานศักย์ใน 2 มิติ



รูปที่ 2.11 : ภาพประกอบของเส้นคอนทัวร์ของพลังงานศักย์ใน 2 มิติ

ความแปรปรวน σ^2 ของตำแหน่งอนุภาคคือ [19]

$$\sigma^2 = \langle x^2 \rangle - \mu^2 \quad (2.37)$$

เนื่องจากการเคลื่อนที่แบบบราวน์ทำให้ค่าเฉลี่ย μ มีค่าเป็น 0 ทำให้ได้สมการ คือ

$$\sigma^2 = \langle x^2 \rangle \quad (2.38)$$

จากทฤษฎีบทการแบ่งเท่ากัน ที่เปรียบเทียบระหว่างพลังงานศักย์กับพลังงานความร้อนจะ
ได้ว่า [11]

$$\frac{1}{2}k\langle x^2 \rangle = \frac{1}{2}k_B T \quad (2.39)$$

นำสมการ (2.38) มาแทนในสมการ (2.39) จะได้

$$\frac{1}{2}k\sigma^2 = \frac{1}{2}k_B T \quad (2.40)$$

ดังนั้นจะทำให้ได้ความแปรปรวน σ^2 คือ

$$\sigma^2 = \frac{k_B T}{k} \quad (2.41)$$

โดยสมการ (2.41) คือ ความแปรปรวนของตำแหน่งของอนุภาคภายใต้คีมจับเชิงแสงที่มีความ
ความแข็งตึง k

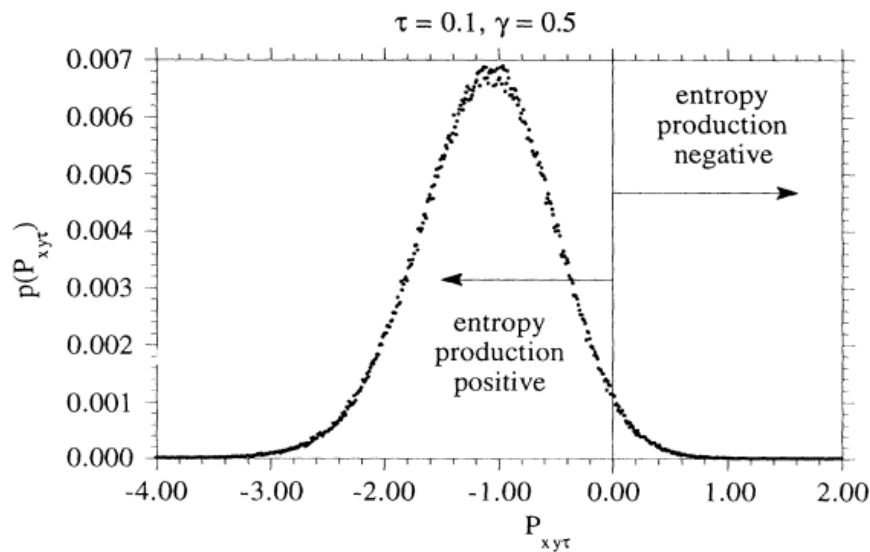
เมื่อแทนพลังงานศักย์ $U(x)$ ในสมการ 2.33 สมการล่องจี้แรงแบบโอเวอร์แดมป์ใน 1
มิติ เขียนได้เป็น

$$\frac{dx}{dt} = -\frac{k}{\gamma}x + \sqrt{\frac{2k_B T}{\gamma}}W(t) \quad (2.42)$$

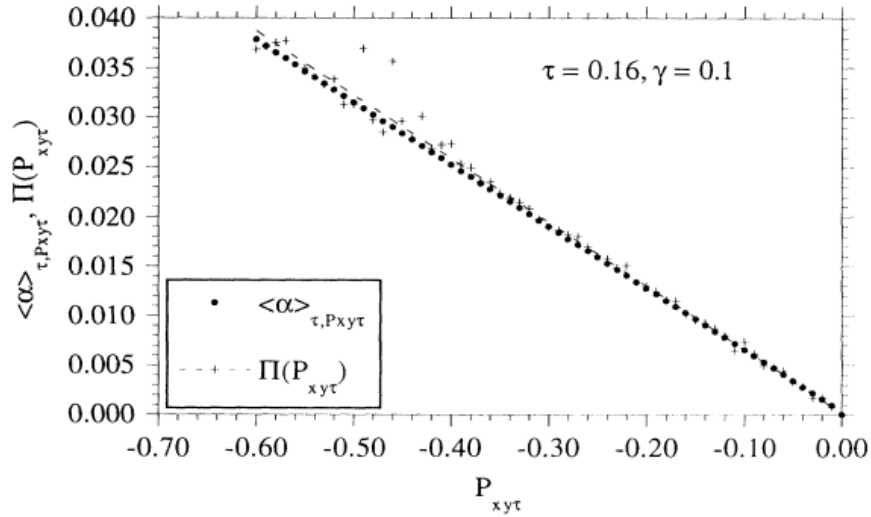
2.5. ทฤษฎีบทความผันผวน (The Fluctuation Theorem)

ในปี ค.ศ. 1993 อีแวนส์ (Evans) โคเฮน (Cohen) และมอร์ริส (Morris) [20]
เสนอสมการที่ค่อนข้างทั่วไปสำหรับลอการิทึม (logarithm) ของอัตราส่วนความน่าจะเป็น
(probability ratio) ที่อยู่ในสภาวะไม่สมดุลของสถานะคงตัว (non-equilibrium steady

state) โดยฟลักซ์การสูญเสียเฉลี่ยตามเวลา (time-averaged dissipative flux) จะมีค่าเป็น $J_+(t)$ และ มีค่าตรงข้ามเป็น $J_-(t) = -J_+(t)$ ซึ่งทำให้เราได้รู้สูตรสำหรับ $\ln[p(J_+(t))/p(J_-(t))]$ ซึ่งเป็นมาตรวัดธรรมชาติที่ไม่แปรเปลี่ยน (natural invariant measure) สมการนี้ใช้อธิบายความน่าจะเป็นสำหรับระบบจำกัด (finite system) และสำหรับเวลาจำกัด (finite time) ที่ฟลักซ์การสูญเสียไหลในทิศทางย้อนกลับ (reverse direction) จะขัดกับกฎข้อที่สองของอุณหพลศาสตร์ ดังรูป 2.12-2.13 สมการนี้ถูกเรียกว่า ทฤษฎีบทความผันผวน (Fluctuation Theorem (FT)) [21]



รูปที่ 2.12 : การแจกแจงความน่าจะเป็นของ $P_{xy\tau}$ ขององค์ประกอบ xy ของเทนเซอร์ความดัน (pressure tensor) สำหรับช่วงเวลา τ สำหรับค่า $P_{xy\tau} > 0$ ที่เป็นบวกแสดงการผลิตเอนโทรปีเป็นค่าลบ ซึ่งขัดกับกฎข้อที่สองของอุณหพลศาสตร์ [20]



รูปที่ 2.13 : อัตราส่วนความน่าจะเป็นเชิงลอการิทึม (logarithmic probability ratio)

$$\Pi(P_{xy\tau}) = \ln [p(P_{xy\tau}) / p(-P_{xy\tau})] / 2N\tau \text{ เป็นฟังก์ชันของ } P_{xy\tau} \text{ [20]}$$

2.5.1. ฟังก์ชันการสูญเสีย (The Dissipation Function)

ฟังก์ชันการสูญเสียเป็นมาตรวัดของระบบที่ผันกลับไม่ได้ เขียนอยู่ในเทอมของอัตราส่วนความหนาแน่นความน่าจะเป็น (probability density) ของการสังเกตเส้นทาง (trajectory) ในช่วงเวลา t กับเส้นทางย้อนกลับ (anti-trajectory) [22] ฟังก์ชันการสูญเสียจะเกี่ยวข้องกับผลผลิตเอนโทรปี (entropy production) ซึ่งการเคลื่อนที่ของระบบจะสอดคล้องกับกฎข้อที่สองที่ให้ฟังก์ชันการสูญเสียมีค่าเป็นบวก (positive dissipation function) คล้ายคลึงกับการเพิ่มผลผลิตเอนโทรปี ส่วนฟังก์ชันการสูญเสียที่มีค่าเป็นบวก (negative dissipation function) แสดงถึงการเคลื่อนที่ตรงข้ามกับกฎข้อที่สองผลผลิตเอนโทรปีมีค่าลดลง [23]

กำหนดให้ $M(t)$ แทนจำนวนสมาชิกทั้งหมดขององชอมเบล (ensemble) ในปริมาตรปริภูมิเฟส (phase space volume) ใด ๆ $V(\Gamma)$ ที่เวลา t และให้ $f(\Gamma, t)$ แทนความหนาแน่นปริภูมิเฟส (phase space density) ดังนั้น [24]

$$M(t) = \int_{V_\Gamma} d\Gamma f(\Gamma, t) \quad (2.43)$$

เมื่อ $f(\Gamma, t)$ คือ ความหนาแน่นที่ตำแหน่งปริภูมิเฟส Γ ที่เวลา t เราจะได้ว่า

$$\frac{dM(t)}{dt} = \int_{V_r} d\Gamma \frac{\partial f(\Gamma, t)}{\partial t} \quad (2.44)$$

เราอาจพิจารณาว่าบางสมาชิกขององชอมเบิ้ลไหลออกจากพื้นผิว S_r รอบ ๆ ปริมาตรดังกล่าว เนื่องจากสมาชิกในองชอมเบิ้ลเป็นปริมาณอนุรักษ์ ทำให้เราได้ว่า

$$\frac{dM(t)}{dt} = - \int_{S_r} dS_\Gamma f(\Gamma, t) \dot{\Gamma}(\Gamma, t) \quad (2.45)$$

เมื่อใช้ทฤษฎีไดเวอร์เจนซ์ (divergence theorem) จะได้

$$\frac{dM(t)}{dt} = - \int_{V_r} d\Gamma \frac{\partial}{\partial \Gamma} [\dot{\Gamma} f(\Gamma, t)] \quad (2.46)$$

เนื่องจากสมการ (2.44) และสมการ (2.46) เป็นจริงสำหรับปริมาตรใด ๆ จะได้ว่า

$$\frac{\partial f(\Gamma, t)}{\partial t} = - \frac{\partial}{\partial \Gamma} \cdot [\dot{\Gamma} f(\Gamma, t)] \quad (2.47)$$

สมการ (2.47) คือ สมการความต่อเนื่องของปริภูมิเฟส (phase space continuity equation) หรือเรียกว่าสมการลิวิลล์ (Liouville equation) [25]

เราสามารถเขียนอนุพันธ์ของ $f(\Gamma, t)$ เทียบกับเวลาได้ว่า

$$\frac{df(\Gamma, t)}{dt} = \frac{\partial f}{\partial t} + \dot{\Gamma} \cdot \frac{\partial f}{\partial \Gamma} \quad (2.48)$$

เมื่อใช้สมการ (2.47) เราสามารถเขียนเทอมอนุพันธ์ใหม่ได้ว่า

$$\frac{df(\Gamma, t)}{dt} = - \frac{\partial}{\partial \Gamma} \cdot [\dot{\Gamma} f(\Gamma, t)] + \dot{\Gamma} \cdot \frac{\partial f}{\partial \Gamma} = -f(\Gamma, t) \frac{\partial}{\partial \Gamma} \cdot \dot{\Gamma} = -f(\Gamma, t) \Lambda(\Gamma) \quad (2.49)$$

เมื่อ

$$\Lambda(\Gamma) \equiv \frac{\partial}{\partial \Gamma} \cdot \dot{\Gamma} \quad (2.50)$$

คือ ตัวแปรการบีบอัดของปริภูมิเฟส (phase space compression factor) ผลเฉลยทั่วไป

ของสมการ (2.49) คือ

$$f(\Gamma(t), t) = \exp \left[- \int_0^t ds \Lambda(\Gamma(s)) \right] f(\Gamma(0), 0) \quad (2.51)$$

พิจารณาปริมาตรปริภูมิเฟสเริ่มต้น $\delta V_\Gamma(\Gamma(0), 0)$ น้อย ๆ รอบจุด $\Gamma(0)$ เมื่อจุดนี้เปลี่ยนไปเป็น $\Gamma(t)$ ที่เวลา t จำนวนสมาชิกภายในปริมาตร $\delta V_\Gamma(\Gamma(t), t)$ จะเท่ากัน

$$f(\Gamma(0), 0) \delta V_\Gamma(\Gamma(0), 0) = f(\Gamma(t), t) \delta V_\Gamma(\Gamma(t), t) \quad (2.52)$$

ทำให้เขียนได้ว่า

$$\frac{\delta V_\Gamma(\Gamma(0), 0)}{\delta V_\Gamma(\Gamma(t), t)} = \frac{f(\Gamma(t), t)}{f(\Gamma(0), 0)} = \exp \left[- \int_0^t ds \Lambda(\Gamma(s)) \right] \quad (2.53)$$

แสดงให้เห็นว่า

$$\delta V_\Gamma(\Gamma(t), t) = \exp \left[\int_0^t \Lambda(\Gamma(s)) ds \right] \delta V_\Gamma(\Gamma(0), 0) \quad (2.54)$$

ฟังก์ชันเอ็กซ์โพเนนเชียลที่อยู่ทางด้านขวาของสมการ (2.54) ทำให้ปริมาตรปริภูมิเฟสเปลี่ยนแปลงตามวิธีการเคลื่อนที่จาก $\Gamma(0)$ ถึง $\Gamma(t)$

พิจารณาความน่าจะเป็น $P(\delta V_\Gamma(\Gamma(t), t))$ ที่สถานะ $\Gamma(t)$ จะถูกสังเกตภายในปริภูมิเฟสที่ปริมาตรน้อย ๆ ที่เวลา t จะได้ว่า [25]

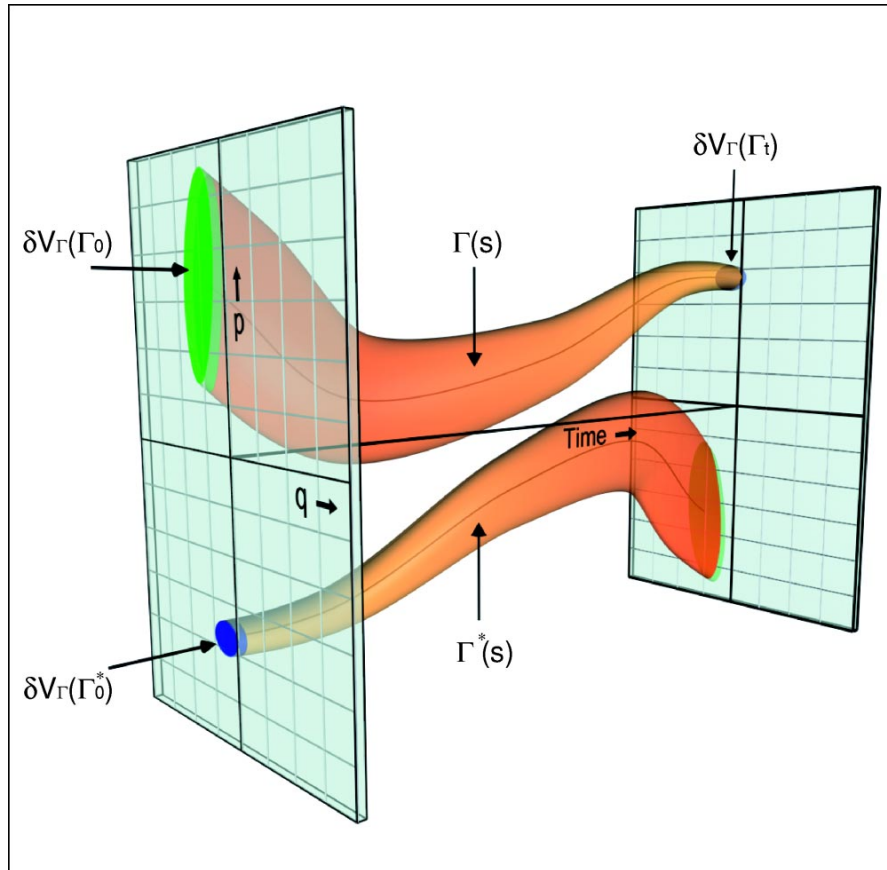
$$P(\delta V_\Gamma(\Gamma(t), t)) = f(\Gamma(t), t) \delta V_\Gamma(\Gamma(t), t) \quad (2.55)$$

สำหรับทุกเส้นทางที่เริ่มต้นที่ $\Gamma(0) \equiv \{(q_0, p_0)\}$ และสิ้นสุดที่ $\Gamma(t) \equiv \{(q_t, p_t)\}$ เมื่อ q_t แทนตำแหน่ง และ p_t แทนโมเมนตัมของอนุภาคที่เวลา t ตามลำดับ ในระบบที่มีพลศาสตร์ผันกลับได้ (reversible dynamics) กำหนดให้ $\Gamma^*(t)$ แทนเส้นทางย้อนกลับ เคลื่อนที่จากสถานะเริ่มต้น $(q_t, -p_t)$ ถึงสถานะสุดท้าย $(q_0, -p_0)$ ดังนั้น $\Gamma(t)$ และ $\Gamma^*(t)$ เป็นคู่ของเส้นทางสังยุค (conjugate trajectory) ซึ่งสัมพันธ์กันด้วยการผกผันเวลา (time reversal)

พิจารณาเซตของเส้นทาง ซึ่งเริ่มตอนที่เวลา $t = 0$ ภายในปริมาตรเล็กน้อยมาก (infin-

itesimal volume) $\delta V_\Gamma(\Gamma(0)) \equiv \delta q_0 \delta p_0$ รอบ ๆ สถานะเริ่มต้น (q_0, p_0) และเซตของเส้นทางสังยุคที่เริ่มภายในองค์ประกอบปริมาตร $\delta V_\Gamma(\Gamma^*(0)) \equiv \delta q_t \delta p_t$ รอบ ๆ สถานะ $(q_t, -p_t)$ รูปที่ 2.14 แสดงให้เห็นเซตเหล่านี้ของเส้นทางในปริภูมิ (q, p) โดยทุกเส้นทางเริ่มต้นที่อยู่ภายใน $\delta V_\Gamma(\Gamma(0))$ จะมีเส้นทางสังยุคเริ่มต้นใน $\delta V_\Gamma(\Gamma^*(0))$ เสมอ [22, 23] พิจารณาอัตราส่วนของความน่าจะเป็นที่เวลาเริ่มต้น $t = 0$ ที่จะเจอเส้นทางปกติในปริมาตร $\delta V_\Gamma(\Gamma(0))$ (พื้นที่สีเขียวในรูป 2.14) ต่อเส้นทางย้อนกลับในปริมาตร $\delta V_\Gamma(\Gamma^*(0))$ (พื้นที่สีน้ำเงินในรูป 2.14) [26]

$$\frac{P(\delta V_\Gamma(\Gamma(0)), 0)}{P(\delta V_\Gamma(\Gamma^*(0)), 0)} = \frac{f(\Gamma(0), 0) \delta V_\Gamma(\Gamma(0))}{f(\Gamma^*(0), 0) \delta V_\Gamma(\Gamma^*(0))} \quad (2.56)$$



รูปที่ 2.14 : ภาพประกอบของเซตของเส้นทางที่ใกล้เคียงกัน เริ่มต้นในปริมาตร $\delta V_\Gamma(\Gamma(0))$ (ท่อด้านบน) สอดคล้องกับเซตของเส้นทางผันกลับที่เริ่มต้นใน $\delta V_\Gamma(\Gamma(0)^*)$ (ท่อด้านล่าง) [22, 27]

สังเกตว่าปริมาตร $\delta V_\Gamma(\Gamma^*(0)) = \delta V_\Gamma(\Gamma(t))$ มีค่าเท่ากัน นอกจากนี้ $f(\Gamma^*(0), 0) = f(\Gamma(t), 0)$ ไม่มีการเปลี่ยนแปลงภายใต้การแปลงโมเมนตัม ดังนั้นจากสมการ (2.53) จะ

ได้ว่า

$$\begin{aligned} \frac{P(\delta V_\Gamma(\Gamma(0)), 0)}{P(\delta V_\Gamma(\Gamma^*(0)), 0)} &= \frac{f(\Gamma(0), 0)\delta V_\Gamma(\Gamma(0))}{f(\Gamma(t), 0)\delta V_\Gamma(\Gamma(t))} \\ &= \frac{f(\Gamma(0), 0)}{f(\Gamma(t), 0)} \exp \left[- \int_0^t ds \Lambda(\Gamma(s)) \right] \end{aligned} \quad (2.57)$$

เรานิยามฟังก์ชันการสูญเสียได้ว่า [26]

$$\exp(\Omega_t(\Gamma)) \equiv \frac{f(\Gamma(0), 0)}{f(\Gamma(t), 0)} \exp \left[- \int_0^t ds \Lambda(\Gamma(s)) \right] \quad (2.58)$$

ทำให้เขียนสมการ (2.57) ได้ใหม่ว่า [26, 28, 29]

$$\begin{aligned} \Omega_t(\Gamma) &= \ln \left[\frac{P(\delta V_\Gamma(\Gamma(0)), 0)}{P(\delta V_\Gamma(\Gamma^*(0)), 0)} \right] \\ &= \ln \left(\frac{f(\Gamma(0), 0)}{f(\Gamma(t), 0)} \right) - \int_0^t ds \Lambda(\Gamma(s)) \end{aligned} \quad (2.59)$$

ถ้าเรานิยามฟังก์ชันสูญเสียเฉลี่ยต่อเวลา $\bar{\Omega}_\tau$ และฟังก์ชันสูญเสียต่อเวลาใด ๆ $\Omega(\Gamma(t))$ จะได้ว่า [22, 26]

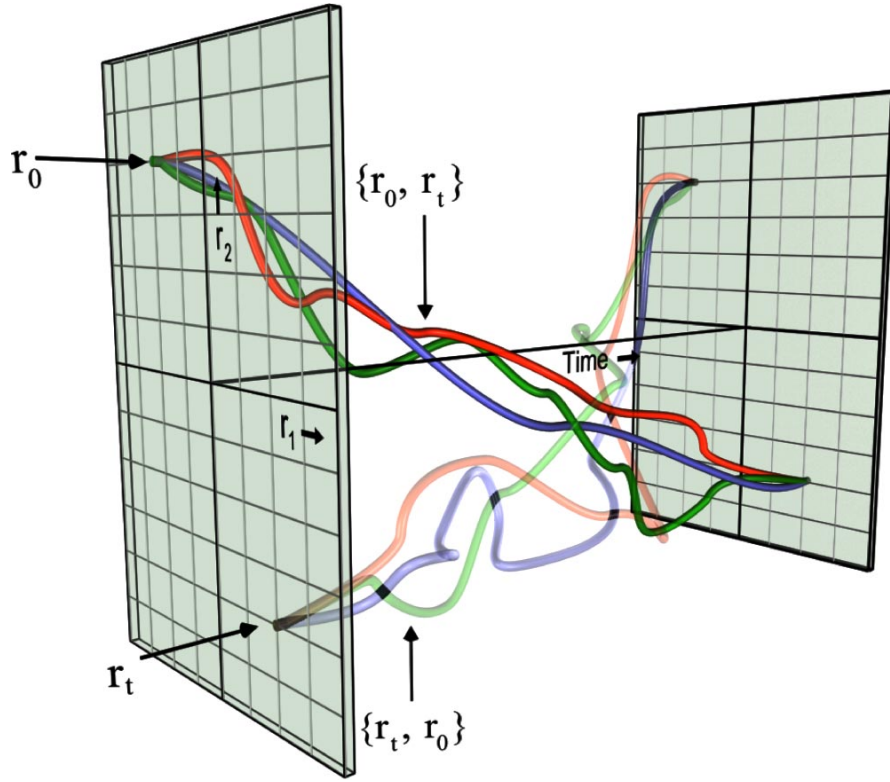
$$\Omega_\tau \equiv \bar{\Omega}_\tau \tau = \int_0^\tau dt \Omega(\Gamma(t)) \quad (2.60)$$

2.5.2. ฟังก์ชันการสูญเสียสำหรับสมการลิ่งจิว์

(Langevin Derivation of Dissipation Function)

สำหรับระบบคอลลอยด์ (colloidal system) ที่สถานะของระบบแทนด้วยตำแหน่ง $r_t = (q_t, p_t)$ เพียงอย่างเดียว การเคลื่อนที่ของอนุภาคเป็นไปตามสมการลิ่งจิว์ เส้นทางการเคลื่อนที่ของอนุภาคจะไม่ได้มีเส้นทางเพียงเส้นทางเดียวที่เริ่มต้นที่สถานะ r_0 และสิ้นสุดที่สถานะ r_t แต่มีได้หลากหลายเส้นทางเนื่องจากระบบมีอันตรกิริยากับสิ่งแวดล้อม ทำให้เกิดเส้นทางการเคลื่อนที่แบบสุ่ม (stochastic trajectories) กำหนดให้ $\{r_0, r_t\}$ แทนเซตของเส้นทางสุ่มเหล่านี้ที่ค่อย ๆ พัฒนาจาก r_0 ถึง r_t และกำหนดให้ $\{r_t, r_0\}$ แทนเซตสังยุค (conjugate set) ของเส้นทางย้อนกลับ (backward trajectories) ที่ค่อย ๆ พัฒนา

จาก r_t ถึง r_0 ซึ่งเซตของเส้นทางไปข้างหน้า (forward trajectories) $\{r_0, r_t\}$ จะสอดคล้องกับเซตของเส้นทางย้อนกลับ $\{r_t, r_0\}$



รูปที่ 2.15 : ภาพประกอบของเซตเส้นทางสุ่มที่เริ่มต้นที่ r_0 และสิ้นสุดที่ r_t แทนด้วย $\{r_0, r_t\}$ สอดคล้องกับเซตเส้นทางย้อนกลับ $\{r_t, r_0\}$ [22, 27]

ให้ $\delta V_{r,r}(\{r_0, r_t\})$ แทนปริมาตรน้อย ๆ ของเซตของเส้นทางที่เริ่มต้นที่ r_0 และสิ้นสุดที่ r_t หรือ $\{r_0, r_t\}$ [22]

$$\delta V_{r,r}(\{r_0, r_t\}) \equiv \lim_{\delta r_0, \delta r_t \rightarrow 0} \delta r_0 \delta r_t \quad (2.61)$$

ความน่าจะเป็นของการสังเกตเส้นทางสุ่มของอนุภาคคอลลอยด์ที่มีตำแหน่งเริ่มต้นระหว่าง r_0 และ $r_0 + \delta r_0$ และตำแหน่งสุดท้ายระหว่าง r_t และ $r_t + \delta r_t$ เขียนได้เป็น

$$P[\delta V_{r,r}(\{r_0, r_t\})] = p(r_0, r_t) \delta r_0 \delta r_t \quad (2.62)$$

เมื่อ $p(r_0, r_t)$ คือ การกระจายตัวความน่าจะเป็น (probability distribution) $p(r_0, r_t)$ และปริมาตรที่พิจารณามีขนาดเท่ากัน $|\delta r_0| = |\delta r_t|$

เราสามารถอธิบายการผันกลับได้ของระบบพลวัตแบบสุ่ม (stochastic dynamics) ใน

ลักษณะเดียวกัน ไม่ขึ้นกับแบบจำลองที่ใช้อธิบายระบบ เมื่อเปรียบเทียบกับสมการที่ (2.59)

$$\Omega_t(r_0, r_t) = \ln \left(\frac{P(\delta V_{r,r}(\{r_0, r_t\}))}{P(\delta V_{r,r}(\{r_t, r_0\}))} \right) = \ln \left(\frac{p(r_0, r_t)}{p(r_t, r_0)} \right) \quad (2.63)$$

สมการที่ (2.63) ใช้ได้ไม่เฉพาะกับอนุภาคคอลลอยด์เดี่ยว ๆ เพียงเท่านั้น แต่ยังสามารถใช้กับระบบอนุภาคคอลลอยด์จำนวนมากได้ ในกรณีนั้น $r \rightarrow \mathbf{r}$ คือ เวกเตอร์ Nd มิติ เมื่อ N คือ จำนวนของอนุภาคคอลลอยด์เคลื่อนที่ใน d มิติ อย่างไรก็ตาม ความหนาแน่นความน่าจะเป็นในสมการที่ (2.63) สามารถอธิบายระบบหลายอนุภาคที่มีความหนาแน่นน้อย ๆ เท่านั้น

2.5.3. ทฤษฎีบทความผันผวนชั่วครู่

(Transient Fluctuation Theorem)

ถัดไปเราแสดงให้เห็นว่าฟังก์ชันการสูญเสีย $\Omega_t(r_0, r_t)$ เป็นไปตามทฤษฎีบทความผันผวน พิจารณาความหนาแน่นความน่าจะเป็น $p(r_0, r_t)$ ที่จะเจอเส้นทางที่ให้ฟังก์ชันการสูญเสีย $\Omega_t = A$ ในช่วงเวลา t เมื่อ A เป็นค่าคงที่ใด ๆ ดังนั้น

$$P(\Omega_t = A) = \int d\mathbf{r}_0 d\mathbf{r}_t \delta[\Omega_t(r_0, r_t) - A] p(r_0, r_t) \quad (2.64)$$

เช่นเดียวกัน จะเจอเส้นทางที่ให้ฟังก์ชันการสูญเสีย $\Omega_t(r_0, r_t) = -A$

$$\begin{aligned} P(\Omega_t = -A) &= \int d\mathbf{r}_0 d\mathbf{r}_t \delta[\Omega_t(r_0, r_t) + A] p(r_0, r_t) \\ &= \int d\mathbf{r}_0 d\mathbf{r}_t \delta[\Omega_t(r_t, r_0) - A] p(r_0, r_t) \end{aligned} \quad (2.65)$$

จากสมการ (2.63) สังเกตว่า $\Omega_t(r_0, r_t) = -\Omega_t(r_t, r_0)$ เมื่อแทนค่า $p(r_0, r_t)$

$$\begin{aligned} P(\Omega_t = -A) &= \int d\mathbf{r}_0 d\mathbf{r}_t \delta[\Omega_t(r_t, r_0) - A] \exp[-\Omega_t(r_t, r_0)] p(r_t, r_0) \\ &= \exp(-A) \int d\mathbf{r}_0 d\mathbf{r}_t \delta[\Omega_t(r_t, r_0) - A] p(r_t, r_0) \end{aligned}$$

$$\frac{P(\Omega_t = -A)}{P(\Omega_t = A)} = \frac{\exp(-A) \int d\mathbf{r}_0 d\mathbf{r}_t \delta[\Omega_t(r_t, r_0) - A] p(r_t, r_0)}{\int d\mathbf{r}_0 d\mathbf{r}_t \delta[\Omega_t(r_0, r_t) - A] p(r_0, r_t)}$$

เนื่องจาก r_0 และ r_t มองเป็นตัวแปรปริพันธ์ใด ๆ ดังนั้น

$$\frac{P(\Omega_t = -A)}{P(\Omega_t = A)} = \exp(-A) \quad (2.66)$$

เป็นไปตามทฤษฎีบทความผันผวน โดยปกติแล้วถ้าสถานะเริ่มต้นของระบบอยู่นอกสมดุล สมการ (2.66) จะเรียกว่าทฤษฎีบทความผันผวนชั่วครู่ ปริมาณ $A = A_t$ ก็จะขึ้นกับเวลา เช่นเดียวกัน ถ้าเราแทน $\Omega_t = \bar{\Omega}_t t$ ทฤษฎีบทความผันผวนชั่วครู่จะเขียนอยู่ในรูป

$$\frac{P(\bar{\Omega}_t = -A)}{P(\bar{\Omega}_t = A)} = \exp(-At) \quad (2.67)$$

จากสมการ (2.66) จะเห็นว่า

$$\begin{aligned} \int dP(\Omega_t = A) \exp(-A) &= \int dP(\Omega_t = -A) \\ \langle \exp(-A) \rangle &= 1 \end{aligned} \quad (2.68)$$

สมการ (2.68) ให้สมการที่สำคัญคือ

$$\langle A \rangle \geq 0 \quad (2.69)$$

ค่าเฉลี่ยของปริมาณ A จะคงที่หรือเพิ่มขึ้นเสมอ ผลลัพธ์นี้มีความทั่วไปเป็นอย่างมาก ใช้ได้กับระบบที่อยู่นอกสมดุล (non-equilibrium) และสามารถนำไปสู่ทฤษฎีอื่น ๆ ที่สำคัญอีกมากมายเช่น อสมการของกฎข้อที่สอง (second law inequality) หรือเอกลักษณ์แบบคาวาซากิ (Kawasaki identity) เป็นต้น [28, 30, 31]

2.5.4. ฟังก์ชันการสูญเสียสำหรับการทดสอบการจับอนุภาค

(The Dissipation Function for Capture Experiment)

อองซอมเบลของเส้นทางสามารถถูกสร้างขึ้นโดยใช้สมการลิ่งจี้แวง [22] จากสมการ (2.42) พิจารณาระบบใน 2 มิติ

$$\gamma \frac{d\mathbf{r}(t)}{dt} = -k_1 \mathbf{r}(t) + \sqrt{2\gamma k_B T} \mathbf{W}(t) \quad (2.70)$$

เมื่อ $\mathbf{r}(t) = (x(t), y(t))$ ตำแหน่งของอนุภาคใน 2 มิติ

กำหนดให้ตำแหน่งในสมดุลเริ่มต้น $\mathbf{r}_0$ มีการกระจายตัวแบบโบลต์ซมันน์ (Boltzmann distribution) ภายใต้คิมจับแสง 2 มิติ ที่มีความแข็งตึง k_0 ดังนั้น

$$P_B(\mathbf{r}_0, k_0) = \left(\frac{k_0}{2\pi k_B T} \right) \exp \left(-\frac{k_0 \mathbf{r}_0^2}{2k_B T} \right) \quad (2.71)$$

ฟังก์ชันกรีน (Green's function) ให้ความน่าจะเป็นของการสังเกตอนุภาคที่ตำแหน่ง $\mathbf{r}_t$ ในกับดักที่มีความแข็งตึง k_1 เมื่อตำแหน่งก่อนหน้าเป็น $\mathbf{r}_0$

$$G(\mathbf{r}_t; \mathbf{r}_0, k_1, t) = \left(\frac{k_1}{2\pi k_B T [1 - \exp(-2t/\tau)]} \right) \exp \left(-\frac{k_1 [\mathbf{r}_t - \mathbf{r}_0 \exp(-t/\tau)]^2}{2k_B T [1 - \exp(-2t/\tau)]} \right) \quad (2.72)$$

เมื่อ $\tau = \gamma/k_1$ คือ เวลาผ่อนคลายลักษณะเฉพาะ (characteristic relaxation time) ของอนุภาคอยู่ในศักย์ฮาร์มอนิก (harmonic potential) ของความแข็งตึง k_1 ฟังก์ชันกรีนมีสมบัติคือ ที่ $t \rightarrow 0$

$$\lim_{t \rightarrow 0} G(\mathbf{r}_t; \mathbf{r}_0, k_1, t) = \delta(\mathbf{r}_t - \mathbf{r}_0)$$

ในลิมิตเมื่อเวลามาก $t \rightarrow \infty$ ฟังก์ชันกรีนลดรูปเป็นการกระจายตัวแบบโบลต์ซมันน์ $P_B(\mathbf{r}_t, k_1)$ ในสมดุลสำหรับอนุภาคในบ่อศักย์ของความแข็งตึง k_1

$$\lim_{t \rightarrow \infty} G(\mathbf{r}_t; \mathbf{r}_0, k_1, t) = \left(\frac{k_1}{2\pi k_B T} \right) \exp \left(-\frac{k_1 \mathbf{r}_t^2}{2k_B T} \right)$$

ดังนั้นความหนาแน่นของความน่าจะเป็นของเส้นทางไปข้างหน้า $p(\mathbf{r}_0, \mathbf{r}_t)$ เป็นผลคูณของความน่าจะเป็นที่อนุภาคที่อยู่ในสภาวะสมดุลที่ตำแหน่ง $\mathbf{r}_0$ ที่ $t = 0$ ภายใต้คิมจับแสงความแข็ง k_0 กล่าวคือ $P_B(\mathbf{r}_0, k_0)$ และความน่าจะเป็นของการสังเกตอนุภาคที่ตำแหน่ง $\mathbf{r}_t$ ที่เวลา t หลังจากความแข็งของกับดักเปลี่ยนแปลงเป็น k_1 โดยมีตำแหน่งเริ่มต้นเป็น $\mathbf{r}_0$ หรือ $G(\mathbf{r}_t; \mathbf{r}_0, k_1, t)$ นั่นคือ

$$p(\mathbf{r}_0, \mathbf{r}_t) = P_B(\mathbf{r}_0, k_0) G(\mathbf{r}_t; \mathbf{r}_0, k_1, t) \quad (2.73)$$

ในทำนองเดียวกัน ความหนาแน่นของความน่าจะเป็นที่สอดคล้องกับเส้นทางย้อนกลับคือ

$$p(\mathbf{r}_t, \mathbf{r}_0) = P_B(\mathbf{r}_t, k_0)G(\mathbf{r}_0; \mathbf{r}_t, k_1, t) \quad (2.74)$$

ทำให้ฟังก์ชันสูญเสียตามสมการ (2.63) สามารถลดรูปอย่างง่ายเป็น

$$\Omega_t(\mathbf{r}_0, \mathbf{r}_t) = \frac{1}{2k_B T} (k_0 - k_1)(\mathbf{r}_t^2 - \mathbf{r}_0^2) \quad (2.75)$$

สังเกตว่า ฟังก์ชันสูญเสีย $\Omega_t(\mathbf{r}_0, \mathbf{r}_t) = -\Omega_t(\mathbf{r}_t, \mathbf{r}_0)$ แต่ละเส้นทางไม่จำเป็นต้องให้ค่าเป็นบวกเสมอไป อาจมีบางเส้นทางที่ให้ค่าเป็นลบ แต่ค่าเฉลี่ยจะมีความมากกว่าหรือเท่ากับศูนย์เสมอ $\langle \Omega_t \rangle \geq 0$ ซึ่งเราจะแสดงให้เห็นในบท ผลการจำลองและการอภิปรายผล

บทที่ 3

วิธีดำเนินการวิจัย

3.1. การเคลื่อนที่แบบบราวน์ 2 มิติ

เริ่มต้นพิจารณาระบบใน 1 มิติ เมื่อรวมแต่ละแรงจากสมการ (2.21) และสมการ (2.10) ทำให้ได้สมการลึองจิแวงคือ

$$\underbrace{m \frac{d^2x}{dt^2}}_{inertia} = - \underbrace{\gamma \frac{dx}{dt}}_{friction} + \underbrace{\sqrt{2\gamma k_B T} W(t)}_{white\ noise} \quad (3.1)$$

เนื่องจากสมการ (2.23) จึงทำให้เทอมความเฉื่อยหายไป [32] จะทำให้สมการเป็นกระบวนการแพร่

$$\gamma \frac{dx}{dt} = \sqrt{2\gamma k_B T} W(t) \quad (3.2)$$

ใช้หลักการผลต่างอันตะ (finite difference) กับกระบวนการวีเนอร $W(t)$ [32, 33] จะได้

$$\gamma \frac{x_i - x_{i-1}}{\Delta t} = \sqrt{2\gamma k_B T} W_i \quad (3.3)$$

สังเกตว่ากระบวนการวีเนอร $W_i \propto 1/\sqrt{\Delta t}$ กำหนดให้

$$w_i = \sqrt{\Delta t} W_i \quad (3.4)$$

เป็นปริมาณที่ไม่มีหน่วย และมีการกระจายแบบปกติ $\mathcal{N}(0, 1)$ แทนสมการ (3.4) ลงไปสมการ (3.3) และจัดรูปสมการใหม่จะได้ว่า

$$x_i = x_{i-1} + \sqrt{\frac{2k_B T \Delta t}{\gamma}} w_i \quad (3.5)$$

หรือเขียนสมการอีกแบบได้เป็น

$$x_i = x_{i-1} + \sqrt{2D\Delta t} w_i \quad (3.6)$$

และสำหรับระบบใน 2 มิติ สามารถเขียนได้ตรงไปตรงมาว่า

$$\begin{aligned} x_i &= x_{i-1} + \sqrt{2D\Delta t} w_{i,x} \\ y_i &= y_{i-1} + \sqrt{2D\Delta t} w_{i,y} \end{aligned} \quad (3.7)$$

เมื่อ D คือ สัมประสิทธิ์การแพร่ มีค่าเป็น $k_B T / \gamma$
 w_i คือ ลำดับของตัวเลขสุ่มแบบเกาส์ (Gaussian random numbers)
 โดยมีคุณสมบัติดังนี้

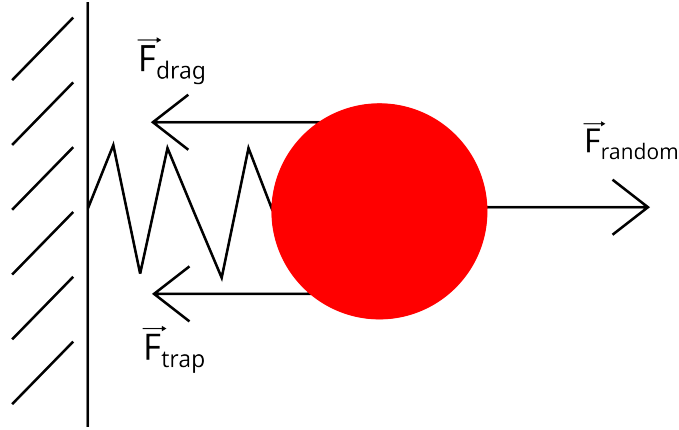
- $\langle w_i \rangle = 0$
- $\langle w_i^2 \rangle = 1$
- $\langle w_i w_j \rangle = 0$ สำหรับ i ต่างจาก j

สังเกตว่าตัวเลขสุ่ม $w_{i,x}$ ในแกน x และ $w_{i,y}$ ในแกน y เป็นอิสระต่อกัน

3.2. คิมจับเชิงแสง

เมื่อเราเพิ่มคิมจับเชิงแสงดังรูป 3.1 จากสมการที่ (2.35) ลงในสมการที่ (3.1) ทำให้สมการได้เป็น

$$\underbrace{m \frac{d^2 x}{dt^2}}_{\text{inertia}} = - \underbrace{\gamma \frac{dx}{dt}}_{\text{friction}} \underbrace{- kx}_{\text{restoring force}} + \underbrace{\sqrt{2\gamma k_B T} W(t)}_{\text{white noise}} \quad (3.8)$$



รูปที่ 3.1 : ทิศทางแรงของสมการลอสจีแวงด้วยคีมจับเชิงแสง

เนื่องจากเราพิจารณาการเคลื่อนที่แบบบราวน์ จึงสามารถละทิ้งเทอมความเฉื่อยได้ ทำให้เราจัดรูปสมการในรูปแบบวิธีเชิงตัวเลขได้เป็น

$$x_i = x_{i-1} - \frac{k}{\gamma} x_{i-1} \Delta t + \sqrt{\frac{2k_B T \Delta t}{\gamma}} w_i \quad (3.9)$$

หรือเขียนสมการอีกแบบได้เป็น

$$x_i = x_{i-1} - \frac{k}{\gamma} x_{i-1} \Delta t + \sqrt{2D\Delta t} w_i \quad (3.10)$$

และสำหรับ 2 มิติ

$$\begin{aligned} x_i &= x_{i-1} - \frac{k}{\gamma} x_{i-1} \Delta t + \sqrt{2D\Delta t} w_{i,x} \\ y_i &= y_{i-1} - \frac{k}{\gamma} y_{i-1} \Delta t + \sqrt{2D\Delta t} w_{i,y} \end{aligned} \quad (3.11)$$

3.3. ฟังก์ชันการสูญสลาย

ความน่าจะเป็นของฟังก์ชันการสูญสลาย $P(\Omega_t)$ ได้จากการหาค่าฟังก์ชันการสูญสลาย Ω_t สำหรับการทดสอบการจับอนุภาคตามสมการที่ (2.75) หลังจากนั้นนำผลที่ได้ทั้งหมดมาความถี่การกระจายตัวของฟังก์ชันการสูญสลาย $\# \Omega_t$ ต่อผลรวมของฟังก์ชันการสูญสลายทั้งหมด $\sum \Omega_t$ เขียนได้ว่า

$$P(\Omega_t) = \frac{\# \Omega_t}{\sum \Omega_t} \quad (3.12)$$

ค่าเฉลี่ยของซอมเบล (ensemble average) ของฟังก์ชันการสูญเสียที่เวลา t ใด ๆ หาได้จากผลรวมตามสมการ

$$\langle \Omega_t \rangle = \sum_i [P(\Omega_i) \Delta x_i] \Omega_i \quad (3.13)$$

เมื่อ Δx_i คือ ขนาดความกว้างของแท่งกราฟ (bin width) ในแนวแกน x

3.4. รูปแบบการดำเนินการ

การศึกษาครั้งนี้มีวัตถุประสงค์เพื่อจำลองเส้นทางการเคลื่อนที่แบบบราวน์และทฤษฎีบทความผันผวนใน 2 มิติโดยการจำลองทางคอมพิวเตอร์ด้วยโปรแกรมไพทอน

3.5. เครื่องมือในการจำลอง

1. เครื่องคอมพิวเตอร์ Intel® Core™ i3-7020U 2.30GHzx2 RAM 4+8 GB
2. โปรแกรมไพทอนเวอร์ชัน 3 (Python 3)
3. ระบบปฏิบัติการลินุกซ์เดเบียน 12 (Debian 12 bookworm)

3.6. ขั้นตอนในการจำลอง

การเคลื่อนที่แบบบราวน์ด้วยสมการกระบวนการแพร่

1. เขียนโปรแกรมไพทอนเพื่อจำลองการเคลื่อนที่แบบบราวน์ใน 2 มิติ ด้วยสมการที่ (3.7)
2. รันโปรแกรมที่ได้จากข้อที่ 1 บนระบบปฏิบัติการลินุกซ์ของคอมพิวเตอร์
3. คำนวณผลเฉลี่ยที่ค่าความหนืดพลวัตต่าง ๆ และ เก็บข้อมูลผลเฉลี่ย
4. นำผลเฉลี่ยที่ได้มาสร้างกราฟด้วยไลบรารี Matplotlib บนโปรแกรมไพทอน

การเคลื่อนที่แบบบราวน์ด้วยคิมจับเชิงแสง

1. เขียนโปรแกรมไพทอนเพื่อจำลองการเคลื่อนที่แบบบราวน์ใน 2 มิติ ด้วยสมการที่ (3.11) และทำการสุ่มตำแหน่งเริ่มต้นด้วยความแปรปรวนจากสมการที่ (2.41)
2. รันโปรแกรมที่ได้จากข้อที่ 1 บนระบบปฏิบัติการลินุกซ์ของคอมพิวเตอร์
3. คำนวณผลเฉลี่ยที่ค่าความหนืดพลวัตต่าง ๆ และ เก็บข้อมูลผลเฉลี่ย

4. นำผลเฉลยที่ได้มาสร้างกราฟด้วยไลบรารี Matplotlib บนโปรแกรมไพทอน พร้อมทั้งทำการเพิ่มเส้นคอนทัวร์ของพลังงานศักย์ด้วยสมการที่ (2.36)

ทฤษฎีบทความผันผวน

1. เขียนโปรแกรมไพทอนเพื่อจำลองการเคลื่อนที่แบบบราวน์ใน 2 มิติ ด้วยสมการที่ (3.11) และทำการสุ่มตำแหน่งเริ่มต้นด้วยความแปรปรวนจากสมการที่ (2.41)
2. รันโปรแกรมที่ได้จากข้อที่ 1 บนระบบปฏิบัติการลินุกซ์ของคอมพิวเตอร์
3. นำผลเฉลยที่ได้ตามเวลาต่าง ๆ มาคำนวณฟังก์ชันการสูญเสียสำหรับการทดลองการจับอนุภาคในสมการที่ (2.75)
4. นำผลเฉลยที่ได้จากข้อที่ 3 มาทำฮิสโตแกรม (histogram) ด้วยไลบรารี NumPy บนโปรแกรมไพทอน และเก็บผลเฉลยที่เวลาต่าง ๆ
5. นำผลเฉลยที่ได้จากข้อที่ 4 มาหาความน่าจะเป็นของฟังก์ชันการสูญเสียจากสมการที่ (3.12)
6. นำผลเฉลยที่ได้จากข้อที่ 5 มาหาค่าเฉลี่ยของขอบเขตของฟังก์ชันการสูญเสียจากสมการที่ (3.13)
7. นำผลเฉลยที่ได้จากข้อที่ 5 มาหาทฤษฎีบทความผันผวนชั่วคราวกับลอการิทึมของทฤษฎีบทความผันผวนชั่วคราวจากสมการที่ (2.66)-(2.69)
8. นำผลเฉลยที่ได้จากข้อที่ 4-7 มาสร้างกราฟด้วยไลบรารี Matplotlib บนโปรแกรมไพทอน

3.7. การวิเคราะห์ข้อมูล

สำหรับการวิเคราะห์ข้อมูล เราจะแยกการวิเคราะห์ออกเป็น 2 ส่วน คือ การเคลื่อนที่แบบบราวน์และทฤษฎีบทความผันผวน

การเคลื่อนที่แบบบราวน์

ในส่วนนี้ เราจะวิเคราะห์การเคลื่อนที่แบบบราวน์จากเส้นทางจากกราฟดังต่อไปนี้

1. สมการกระบวนการแพร่ เราจะวิเคราะห์จากความหนาแน่นพลวัตค่าต่าง ๆ
2. คิมจับเชิงแสง เราจะวิเคราะห์จากความแข็งตึงของกัมมิตค่าต่าง ๆ

ทฤษฎีบทความผันผวน

ในส่วนนี้ เราจะวิเคราะห์ฟังก์ชันการสูญเสียและทฤษฎีบทความผันผวนดังต่อไปนี้

1. **ฟังก์ชันการสูญเสีย** เราจะวิเคราะห์จากฟังก์ชันการสูญเสียกับความน่าจะเป็นของฟังก์ชันการสูญเสียที่มาจากกราฟแบบฮิสโทแกรมและค่าเฉลี่ยของชอมเบิลของฟังก์ชันการสูญเสีย

2. **ทฤษฎีบทความผันผวน** เราจะวิเคราะห์จากทฤษฎีบทความผันผวนชั่วคราวและลอการิทึมของทฤษฎีบทความผันผวนชั่วคราว

3.8. การเปรียบเทียบผลของข้อมูล

การเคลื่อนที่แบบบราวน์

การเปรียบเทียบผลของข้อมูลของการเคลื่อนที่แบบบราวน์ในส่วนของสมการกระบวนการแพร่ จะใช้การเปรียบเทียบเส้นทางในของเหลวที่ความหนืดค่าต่าง ๆ แต่ในคิมจับเชิงแสงจะใช้การเปรียบเทียบเส้นทางในความแข็งตึงของกับดักค่าต่าง ๆ

ทฤษฎีบทความผันผวน

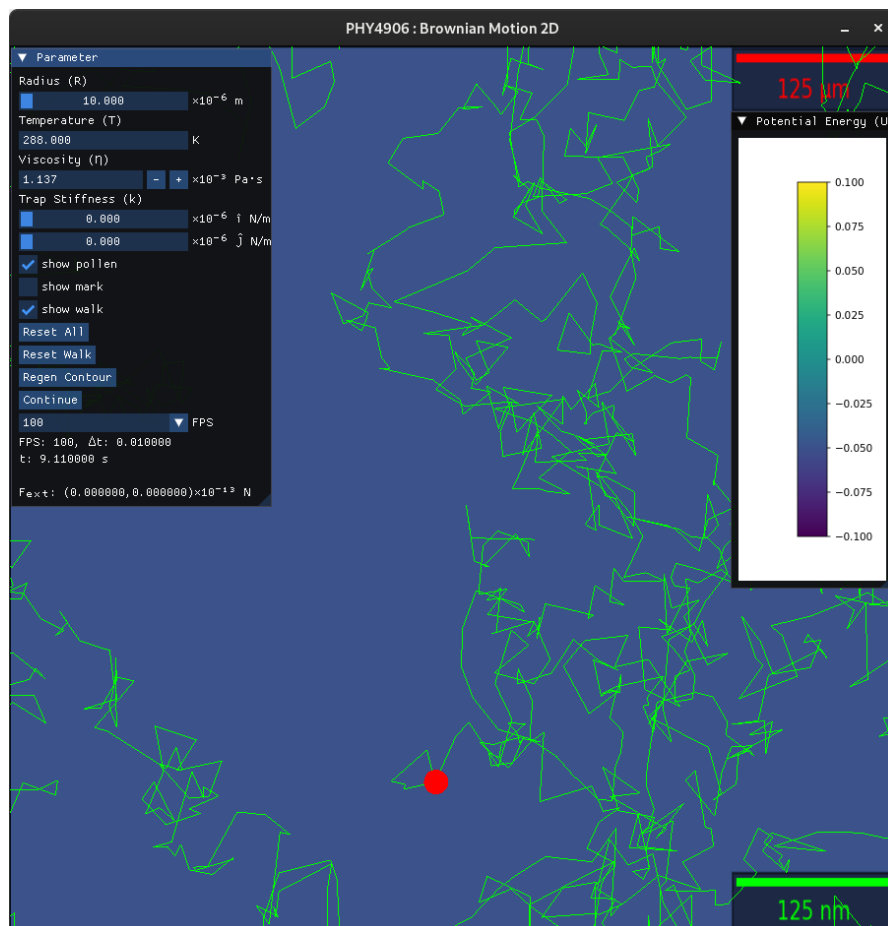
การเปรียบเทียบผลของข้อมูลของฟังก์ชันการสูญเสียกับความน่าจะเป็นของฟังก์ชันการสูญเสียจะใช้วิธีการเปรียบเทียบที่เวลาต่าง ๆ โดยการสร้างกราฟแบบฮิสโทแกรม เพื่อมาเปรียบเทียบ ซึ่งต้องอยู่ในสเกลเดียวกัน แต่ค่าเฉลี่ยของชอมเบิลของฟังก์ชันการสูญเสียจะเทียบแค่กับเวลาโดยจะไม่มีกราฟที่เวลาต่าง ๆ ในส่วนการจำลองจะใช้วิธีสร้างกราฟแบบการกระจาย (scatter plot) แต่การคาดการณ์ (prediction) จะใช้วิธีสร้างกราฟแบบเส้น (line plot) ส่วนทฤษฎีบทความผันผวนชั่วคราวกับลอการิทึมของทฤษฎีบทความผันผวนชั่วคราวจะใช้วิธีการเปรียบเทียบที่เวลาต่าง ๆ

บทที่ 4

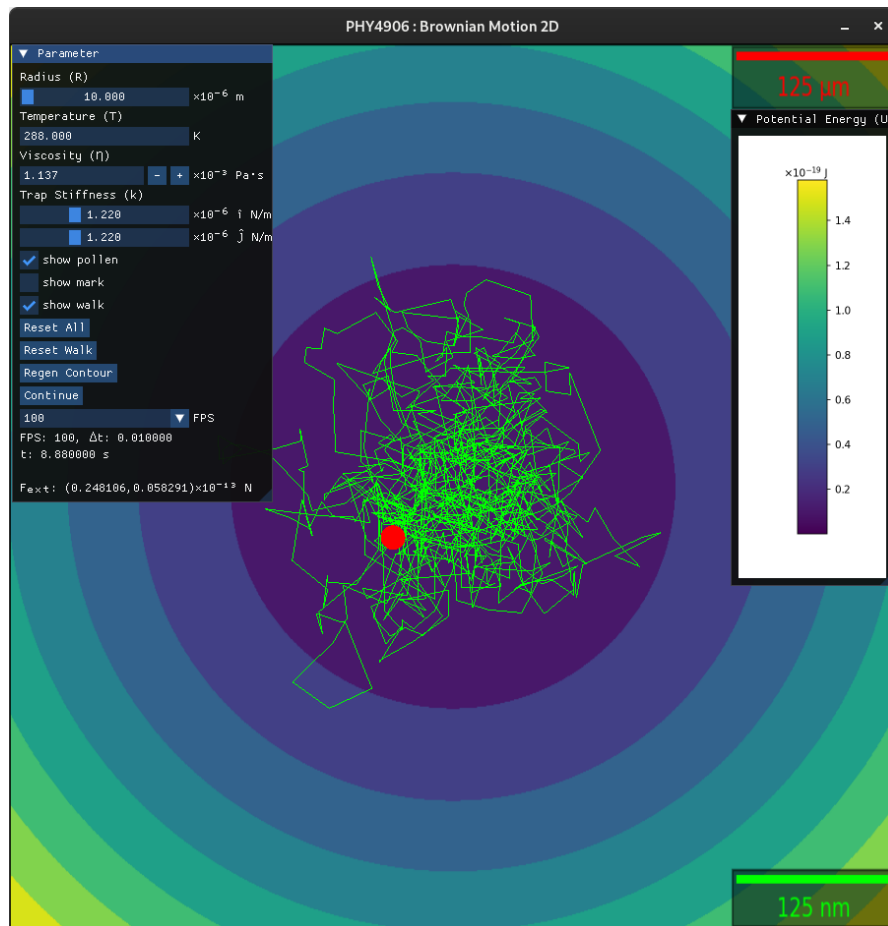
ผลและการอภิปรายผล

4.1. การเคลื่อนที่แบบบราวน์ 2 มิติ

เราสามารถจำลองการเคลื่อนที่แบบบราวน์ 2 มิติ แบบเวลาจริง (real-time) ได้ โดยทำการเขียนโปรแกรมโดยใช้ไลบรารี GLFW, OpenGL, matplotlib, numpy และ imgui_bundle ซึ่งโปรแกรมนี้สามารถปรับตัวแปรเป็นค่าต่างๆ ได้ เช่น รัศมี (R) อุณหภูมิ (T) และความแข็งตึงของก๊าดัก (k) เป็นต้น



รูปที่ 4.1 : ภาพประกอบจินตทัศน์ของเส้นทางการเคลื่อนที่แบบบราวน์ 2 มิติ



รูปที่ 4.2 : ภาพประกอบจินตทัศน์ของเส้นทางการเคลื่อนที่แบบบราวน์ 2 มิติ ด้วยคีมจับเชิงแสง

4.1.1. สมการกระบวนการแพร่

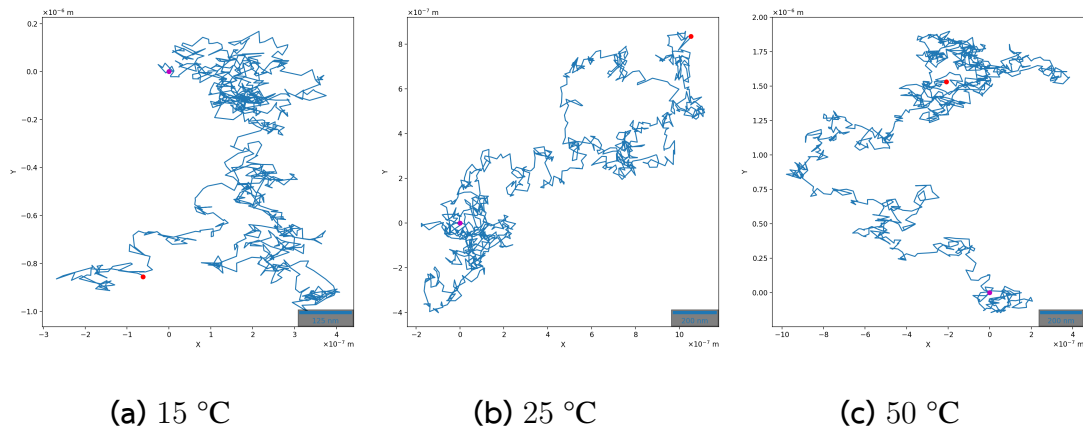
เราจะจำลองการเคลื่อนที่แบบบราวน์ 2 มิติ จากสมการที่ (3.5)-(3.7) โดยเราจะใช้ตัวแปรดังต่อไปนี้ $N = 1000$, $FPS = 100$, $\Delta t = 0.01$ s, $k_B = 1.380649 \times 10^{-23}$ J/K, $R = 10 \times 10^{-6}$ m

โดยเราจะใช้ค่าความหนืดพลวัต η ตามอ้างอิง [34] ดังต่อไปนี้

- น้ำ (water) ที่ $T = 15$ °C = 288.15 K คือ $\eta = 1.139 \times 10^{-3}$ Pa·s
- น้ำ ที่ $T = 25$ °C = 298.15 K คือ $\eta = 0.890 \times 10^{-3}$ Pa·s
- น้ำ ที่ $T = 50$ °C = 323.15 K คือ $\eta = 0.547 \times 10^{-3}$ Pa·s
- น้ำมันมะกอก (olive oil) ที่ $T = 20$ °C = 293.15 K คือ $\eta = 8.4 \times 10^{-2}$ Pa·s
- อะซิโตน (acetone) ที่ $T = 25$ °C = 298.15 K คือ $\eta = 0.316 \times 10^{-3}$ Pa·s
- กลูโคส (glucose) ที่ $T = 30$ °C = 303.15 K คือ $\eta = 6.6 \times 10^{10}$ Pa·s

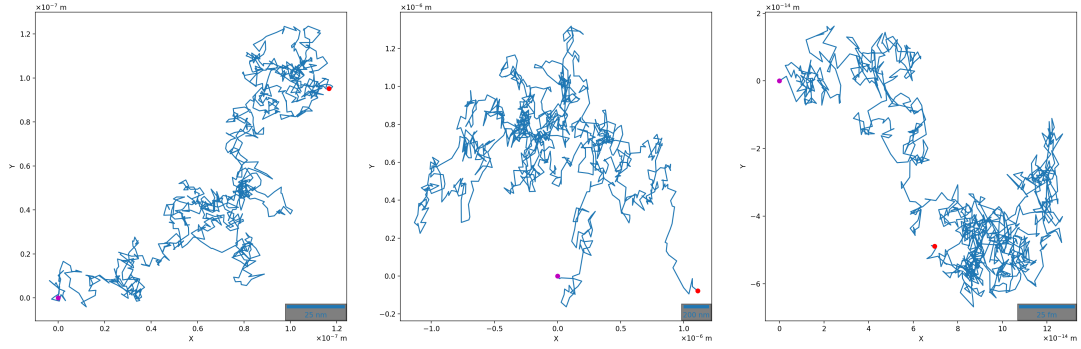
รูปที่ 4.3 ยืนยันความถูกต้องของกระบวนการแพร่ของไอออนสโตน โดยเราได้สังเกต

เห็นว่าการเคลื่อนที่ของละอองเรณูที่ $T = 50\text{ }^{\circ}\text{C}$ เร็วกว่าที่ $T = 25\text{ }^{\circ}\text{C}$ และเร็วกว่าที่ $T = 15\text{ }^{\circ}\text{C}$ เนื่องจากความร้อนทำให้น้ำมีความหนืดพลวัตน้อยลง ทำให้ละอองเรณูเคลื่อนที่ได้เร็วขึ้น ในทางกลับกันความเย็นทำให้น้ำมีความหนืดพลวัตมากขึ้น ดังนั้นละอองเรณูจึงเคลื่อนที่ได้ช้าลง



รูปที่ 4.3 : เส้นทางของการเคลื่อนที่แบบบราวน์ 2 มิติ ในน้ำ โดยตำแหน่งเริ่มต้นแทนด้วยจุดสีม่วงแดง และตำแหน่งสิ้นสุดแทนด้วยจุดสีเขียว

ต่อมาเราทำการจำลองในของเหลวประเภทต่าง ๆ ดังรูปที่ 4.4 ปรากฏว่าละอองเรณูมีความเร็วในการเคลื่อนที่ที่แตกต่างกันไป เช่น กลูโคสมีความหนืดพลวัตที่สูงมาก ทำให้ละอองเรณูเคลื่อนที่ได้ช้ามาก แต่ยังมีกระบวนการแพร่อยู่ ซึ่งต่างจากน้ำมันมะกอกกับอะซิโตน ที่มีความหนืดพลวัตที่สูงแต่ก็ไม่ได้มากเท่ากลูโคส ทำให้ละอองเรณูยังสามารถเคลื่อนที่ได้ดีกว่าในกลูโคสเพราะมีความหนืดพลวัตที่ต่ำ



(a) น้ำมันมะกอก 20 °C

(b) อะซิโตน 25 °C

(c) กลูโคส 30 °C

รูปที่ 4.4 : เส้นทางการเคลื่อนที่แบบบราวน์ 2 มิติ ในของเหลวประเภทต่าง ๆ โดยตำแหน่งเริ่มต้นแทนด้วยจุดสีม่วงแดง และตำแหน่งสิ้นสุดแทนด้วยจุดสีแดง

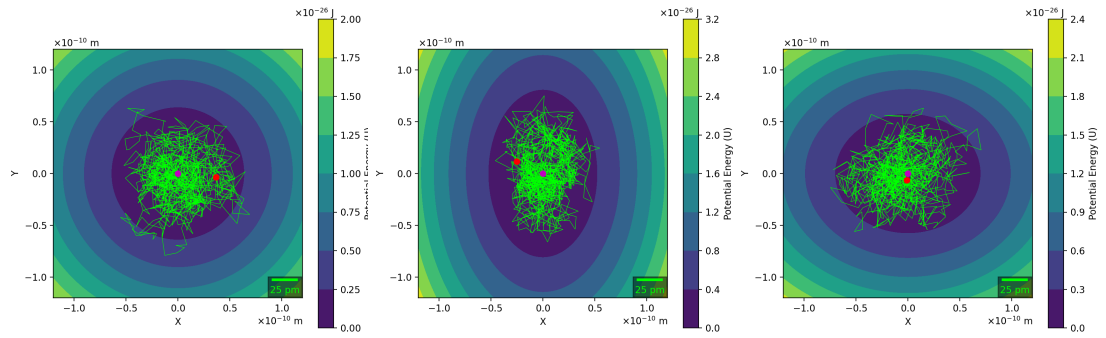
4.1.2. คีมจับเชิงแสง

เราจะจำลองเส้นทางการเคลื่อนที่แบบบราวน์ 2 มิติ ด้วยคีมจับเชิงแสงจากสมการที่ (3.9)-(3.11) และทำการสุ่มตำแหน่งเริ่มต้นด้วยความแปรปรวนจากสมการที่ (2.41) โดยเราจะใช้ตัวแปรดังต่อไปนี้ $N = 10000$, $FPS = 100$, $\Delta t = 0.01$ s, $k_B = 1.380649 \times 10^{-23}$ J/K, $T = 288$ K, $\eta = 1.137 \times 10^{-3}$ Pa·s, $R = 10 \times 10^{-6}$ m

โดยเราจะใช้ค่าความแข็งตึงของกับดักดังต่อไปนี้

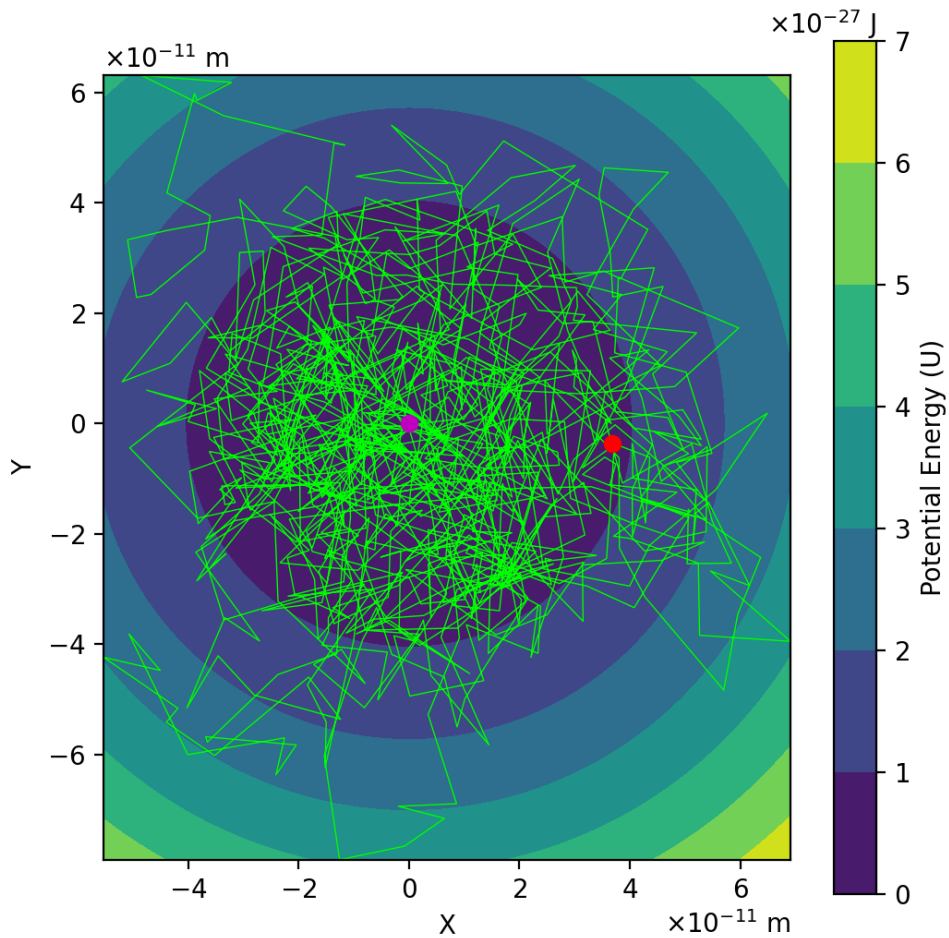
- $(k_x, k_y) = (1.22, 1.22) \times 10^{-6}$ N/m
- $(k_x, k_y) = (2.898, 1.22) \times 10^{-6}$ N/m
- $(k_x, k_y) = (1.22, 1.8) \times 10^{-6}$ N/m

รูปที่ 4.5 ยืนยันความถูกต้องของคีมจับเชิงแสงจากสมการที่ (2.35) โดยสังเกตจากเส้นทางของละอองเรณูบนเส้นคอนทัวร์ของพลังงานศักย์ เมื่อละอองเรณูเคลื่อนที่จะถูกดึงกลับด้วยความแข็งตึงของกับดัก ทำให้ละอองเรณูจะเคลื่อนที่ภายในโซนสีม่วงที่มีพลังงานศักย์ที่ต่ำที่สุด เมื่อเราขยายรูป 4.5(a) เข้าไปดูใกล้ขึ้น ดังรูป 4.6 จะสังเกตเห็นละอองเรณูมีพลังงานศักย์เปลี่ยนไป แต่ส่วนใหญ่ยังเคลื่อนที่อยู่ในพลังงานศักย์ที่ต่ำที่สุด



(a) $(1.22, 1.22) \times 10^{-6} \text{ N/m}$ (b) $(2.898, 1.22) \times 10^{-6} \text{ N/m}$ (c) $(1.22, 1.8) \times 10^{-6} \text{ N/m}$

รูปที่ 4.5 : เส้นทางของการเคลื่อนที่แบบบราวน์ 2 มิติ ด้วยคีมจับเชิงแสง
โดยตำแหน่งเริ่มต้นแทนด้วยจุดสีม่วงแดง และตำแหน่งสิ้นสุดแทนด้วยจุดสีแดง



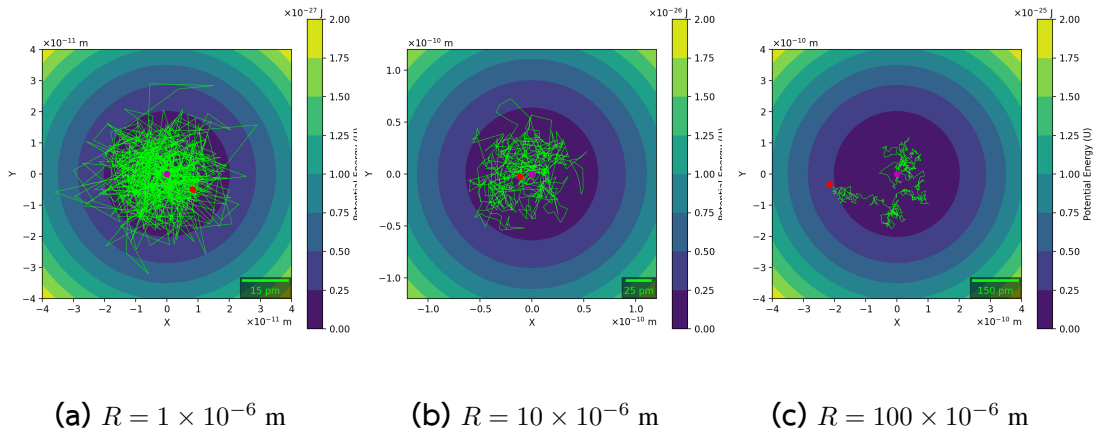
รูปที่ 4.6 : ภาพขยายของเส้นทาง $(1.22, 1.22) \times 10^{-6} \text{ N/m}$

ต่อมาเราจำลองโดยการเปลี่ยนเฉพาะรัศมี R กับ $N = 500$ และใช้ความแข็งดึงของกับดักที่ $(k_x, k_y) = (1.22, 1.22) \times 10^{-6} \text{ N/m}$

โดยเราจะใช้รัศมีดังต่อไปนี้

- $R = 1 \times 10^{-6} \text{ m}$
- $R = 10 \times 10^{-6} \text{ m}$
- $R = 100 \times 10^{-6} \text{ m}$

จากรูปที่ 4.7 เมื่อรัศมีสูงขึ้นทำให้ดักจับได้น้อยจนไม่มีผลกับละอองเรณู และละอองเรณูกลายเป็นการเคลื่อนที่แบบบราวน์ในที่สุด



รูปที่ 4.7 : เส้นทางการเคลื่อนที่แบบบราวน์ 2 มิติ ด้วยคีมจับเชิงแสงที่รัศมีต่าง ๆ โดยตำแหน่งเริ่มต้นแทนด้วยจุดสีม่วงแดง และตำแหน่งสิ้นสุดแทนด้วยจุดสีแดง

4.2. ทฤษฎีบทความผันผวน

4.2.1. ฟังก์ชันการสูญเสีย

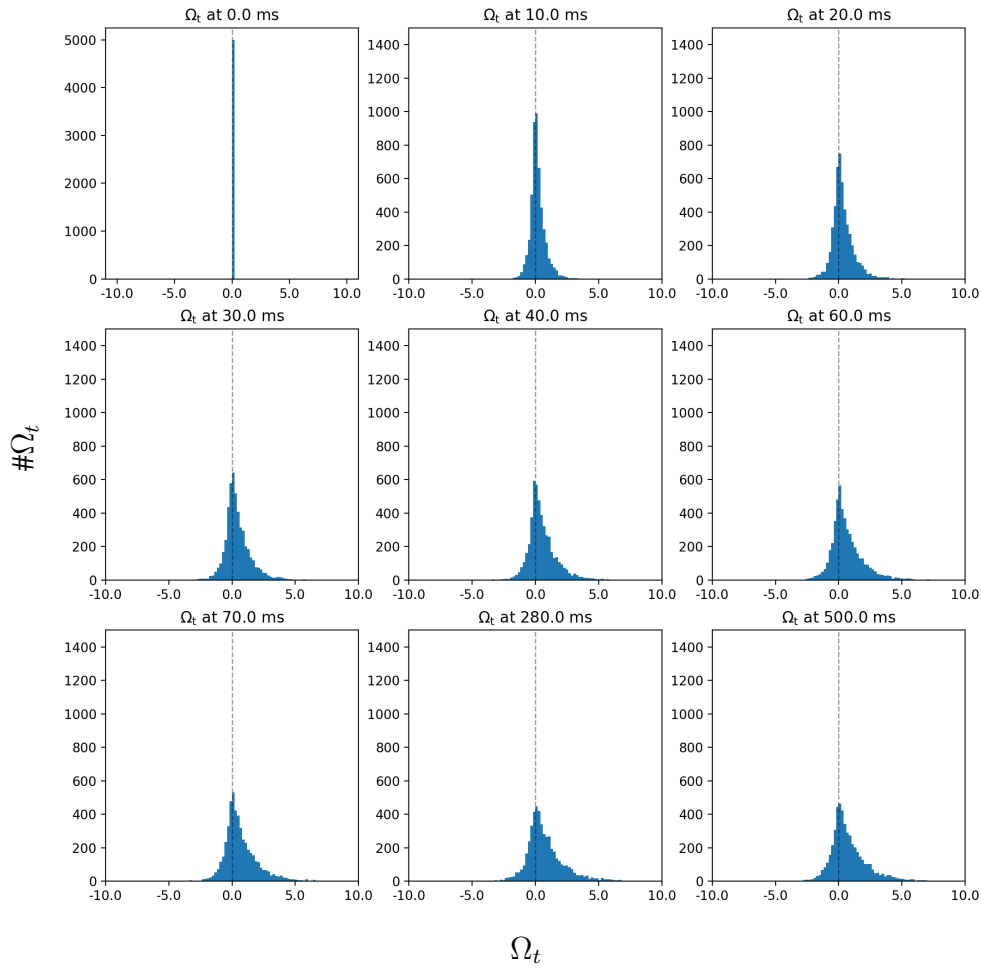
เราจะจำลองการเคลื่อนที่แบบบราวน์ 2 มิติ ด้วยคีมจับเชิงแสงจากสมการที่ (3.9)-(3.11) และทำการสุ่มตำแหน่งเริ่มต้นด้วยความแปรปรวนจากสมการที่ (2.41) เพื่อหาค่าฟังก์ชันการสูญเสียสำหรับการทดลองการจับอนุภาคจากสมการที่ (2.75) โดยเราจะใช้ตัวแปรดังต่อไปนี้ $N = 5000$, $\text{FPS} = 1000$, $\Delta t = 0.001 \text{ s}$, $k_B = 1.380649 \times 10^{-23} \text{ J/K}$, $T = 288 \text{ K}$, $\eta = 1.137 \times 10^{-3} \text{ Pa}\cdot\text{s}$, $R = 10 \times 10^{-6} \text{ m}$, $t = 500 \times 10^{-3} \text{ s}$, $\text{num_step} = 51$, $\text{bin_width} = 0.2$

โดยเราจะใช้ความแข็งดึงของกับดักดังต่อไปนี้

- ความแข็งตึงของกับดักเริ่มต้น $(k_{0,x}, k_{0,y}) = (1.22, 1.22) \times 10^{-6} \text{ N/m}$
- ความแข็งตึงของกับดักใหม่ $(k_{1,x}, k_{1,y}) = (2.9, 2.9) \times 10^{-6} \text{ N/m}$

รูปที่ 4.8 แสดงฮิสโทแกรมของฟังก์ชันการสูญเสียจากสมการที่ (2.75) ซึ่งสร้างขึ้นจากการจำลอง 5000 เส้นทาง ที่เวลาต่าง ๆ เมื่อความแข็งตึงของกับดักเพิ่มขึ้นจาก k_0 ไปยัง k_1 ที่ $t = 0 \text{ ms}$ ฟังก์ชันการสูญเสียที่เกิดจากการสุมตำแหน่งเริ่มต้นจากความแปรปรวน ทำให้ Ω_t จะรวมอยู่ที่ 0 หลังจากเวลาผ่านไป จะค่อย ๆ กระจายออกมา ทำให้ฟังก์ชันการสูญเสียกว้างขึ้นทั้งทางด้านลบและด้านบวก เมื่อเวลาผ่านไปนานขึ้น ค่าจะไปทางบวกมากขึ้น ซึ่งเป็นสิ่งที่บอกได้ว่าระบบไม่สามารถผันกลับได้ เนื่องจากค่าเฉลี่ยฟังก์ชันการสูญเสียมากกว่าศูนย์เสมอ ตามอสมการของกฎข้อที่สอง

เราได้สังเกตเห็นว่าความสูงของฮิสโทแกรมจะลดลงเมื่อเวลามากขึ้น ส่วนความกว้างของฮิสโทแกรมจะมากขึ้นตามเวลา แต่การเปลี่ยนแปลงจะค่อย ๆ ช้าลงที่เวลามาก ๆ แสดงให้เห็นว่าระบบเข้าสู่สภาวะสมดุล

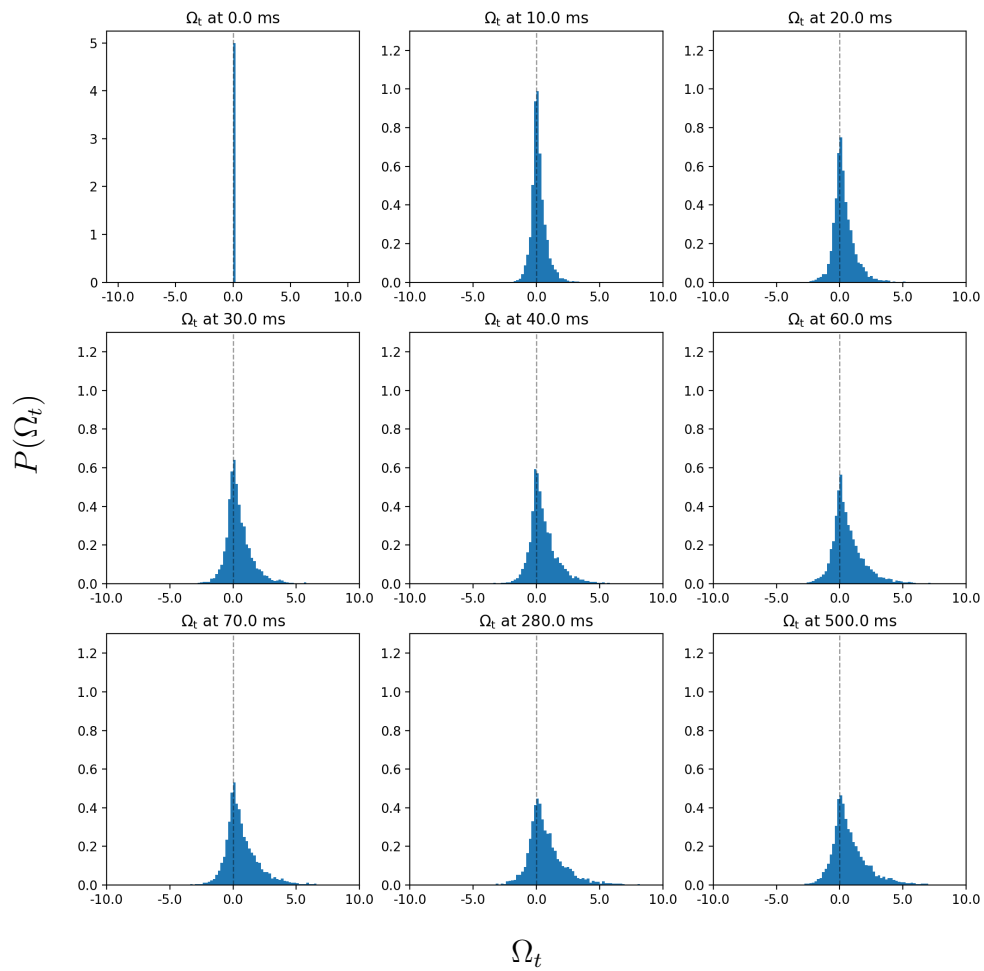


รูปที่ 4.8 : ฟังก์ชันการสูญเสีย

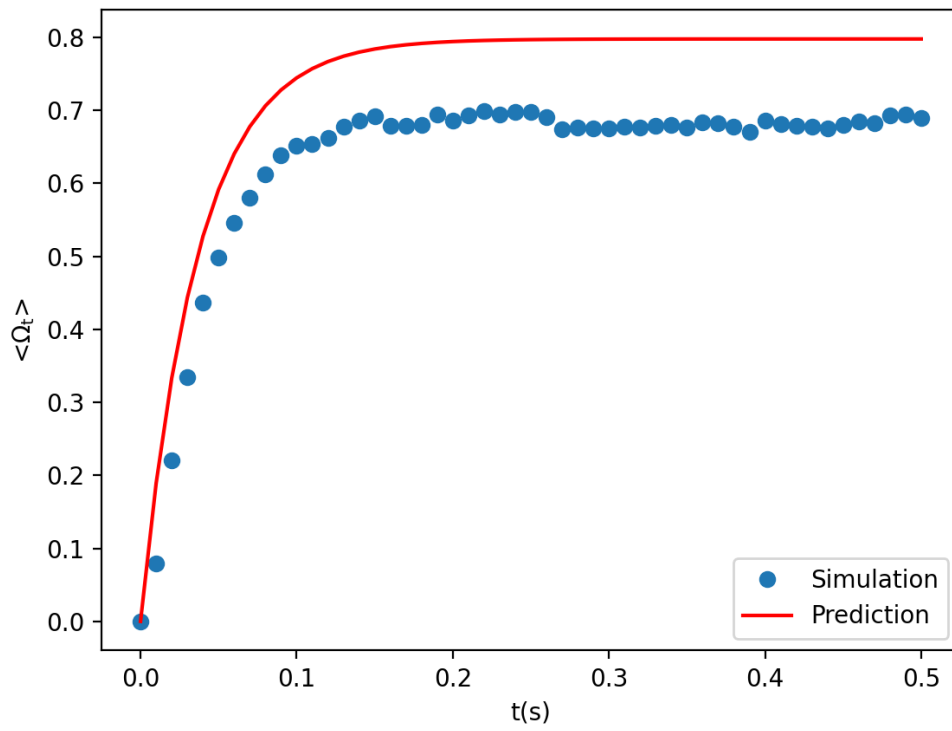
รูปที่ 4.9 คือการนำความถี่ของฟังก์ชันการสูญเสีย $\# \Omega_t$ ไปหาค่า $P(\Omega_t)$ ด้วยสมการที่ (3.12) โดยได้แสดงความน่าจะเป็นของฟังก์ชันการกระจายตัวที่เวลาต่าง ๆ เราได้สังเกตเห็นว่า $P(\Omega_t > 0)$ ที่เวลา $t = 0$ ms ไม่มีการสลายตัวของพลังงาน แต่เมื่อเวลาผ่านไปจะเกิดความแปรปรวนทางอุณหภูมิตัวเอง (thermal fluctuations) [27] ส่งผลทำให้เกิด Ω_t หลายเส้นทางที่เป็นลบ เมื่อเวลาผ่านไปยาวนาน Ω_t หลายเส้นทางที่เป็นลบจะคงที่ เนื่องจากเข้าสู่สถานะสมดุล

รูปที่ 4.10 แสดงกราฟของค่าเฉลี่ยของคอมเบลของฟังก์ชันการสูญเสีย $\langle \Omega_t \rangle$ เทียบกับ $t(s)$ เราได้สังเกตเห็นว่า $\langle \Omega_t \rangle$ เพิ่มขึ้นจากเวลา $t = 0$ s จนเริ่มเข้าสู่สถานะสมดุลที่เวลาประมาณ $t = 0.15$ s โดย $\langle \Omega_t \rangle$ เพิ่มขึ้นอย่างรวดเร็วจาก 0 จนถึง 0.675 ± 0.025 ที่สถานะสมดุล ซึ่ง $\langle \Omega_t \rangle$ จากการจำลอง (simulation) ยังต่างกับการคาดการณ์ (prediction) จาก

สมการ $\langle \Omega_t \rangle = (1/2)(k_0 - k_1)[1/k_1 - 1/k_0][1 - \exp(-2t/\tau)]$ เมื่อ $\tau \equiv \gamma/k_1$ [22] ที่ได้จากการคาดการณ์เมื่อเริ่มเข้าสู่สภาวะสมดุลที่เวลา $t = 0.15$ s แต่ $\langle \Omega_t \rangle$ มีค่าเป็น 0.797 ± 0.001 ทำให้การคาดการณ์ต่างจากการจำลองอย่างเห็นได้ชัด



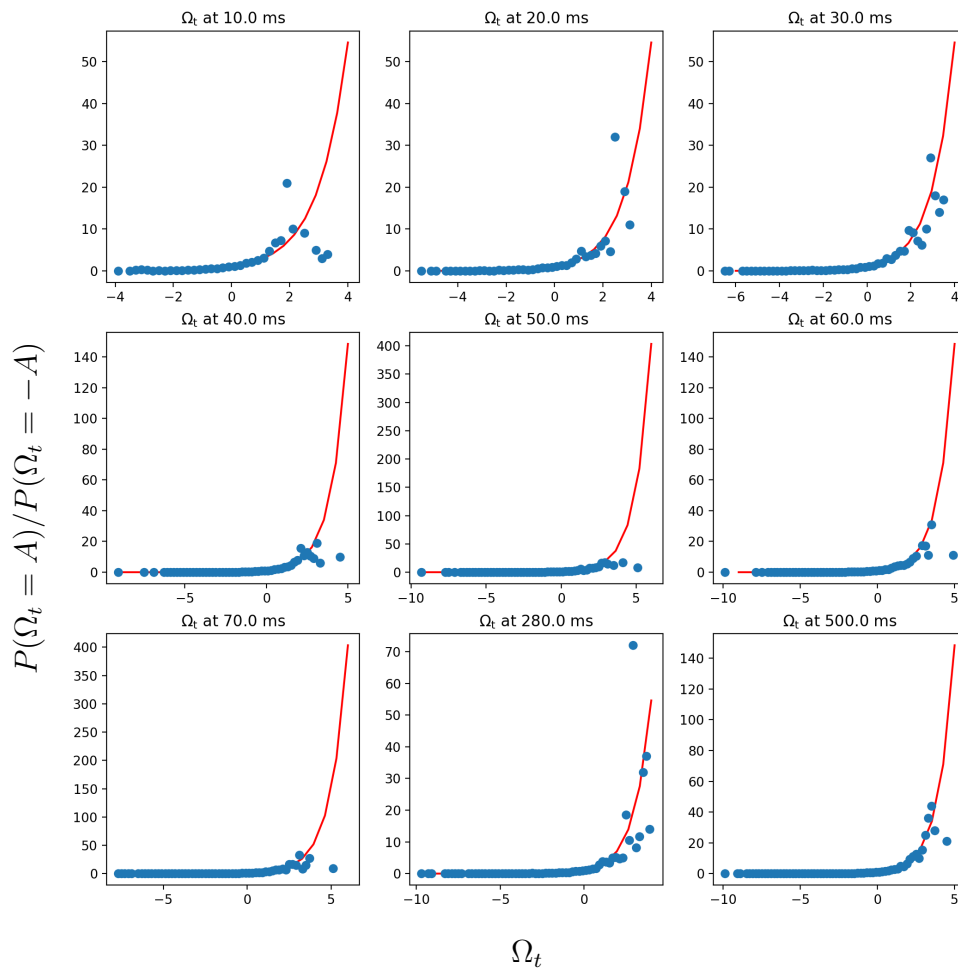
รูปที่ 4.9 : ความน่าจะเป็นของฟังก์ชันการสูญเสีย



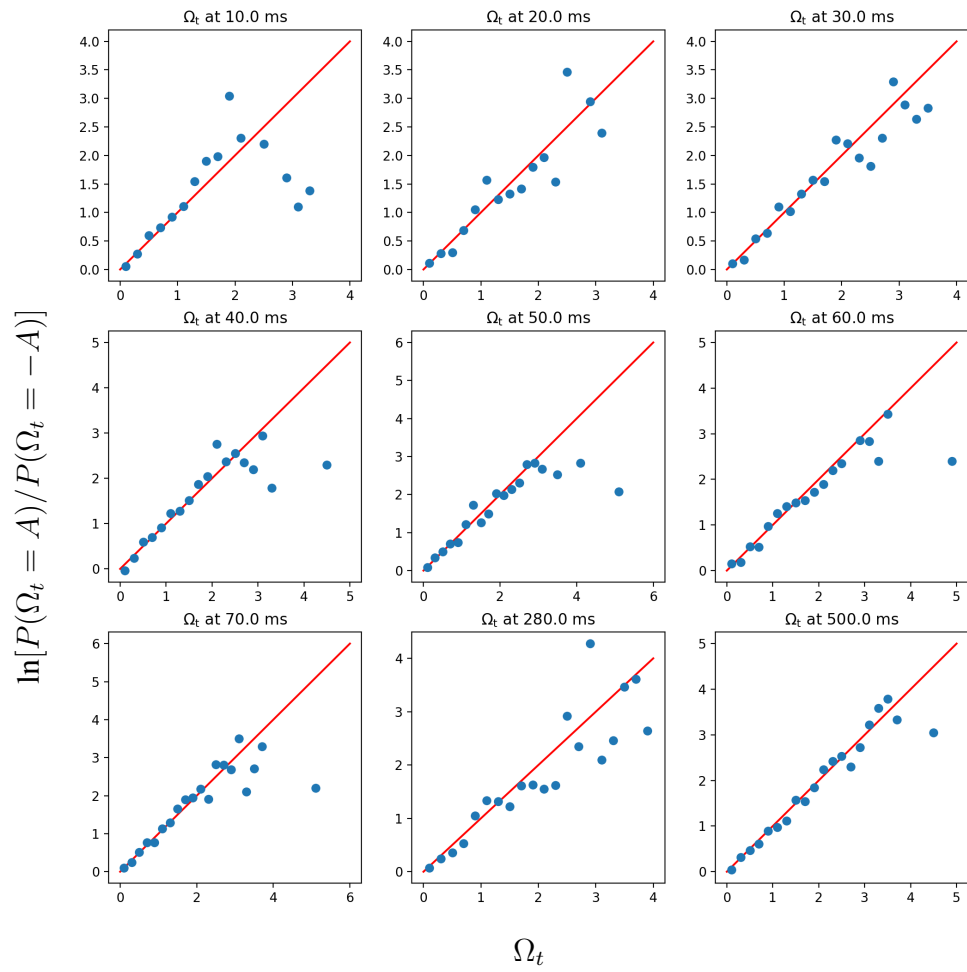
รูปที่ 4.10 : ค่าเฉลี่ยของคอมเบลของฟังก์ชันการสูญเสียเทียบกับเวลา $t(s)$

4.2.2. ทฤษฎีบทความผันผวนชั่วคราว

รูปที่ 4.11 และรูปที่ 4.12 ยืนยันความถูกต้องของทฤษฎีบทความผันผวนชั่วคราวตามสมการที่ (2.66) โดยในรูปที่ 4.11 เราได้สังเกตเห็นว่า Ω_t ในส่วนซ้ายมือ (LHS) เป็นเลขชี้กำลัง (exponential) และในส่วนของเลขชี้กำลังของ Ω_t ในฝั่งขวามือ (RHS) จะลู่เข้าสู่ 5 เมื่อเรานำค่ามาใส่ลอการิทึมดังรูปที่ 4.12 เราจะเห็นส่วนของ RHS เป็นเส้นตรง แต่ในส่วน ของ LHS ภาพรวมยังคงข้างเป็นเส้นตรง แต่ถ้า Ω_t เกิน 2 จะค่อนข้างไม่เป็นเส้นตรงและกระจัดกระจาย



รูปที่ 4.11 : ทฤษฎีบทความผันผวนชั่วคราว



รูปที่ 4.12 : ลอการิทึมของทฤษฎีบทความผันผวนชั่วครู่

บทที่ 5

สรุปผลและข้อเสนอแนะ

5.1. สรุปผล

สมการกระบวนการแพร่

สมการกระบวนการแพร่แสดงให้เห็นถึงการเคลื่อนที่ของละอองเรณู ซึ่งของเหลวแต่ละชนิดมีความสามารถทำให้กระบวนการแพร่มีความแตกต่างกัน โดยละอองเรณูจะเคลื่อนที่ช้าหรือเร็วขึ้นอยู่กับอุณหภูมิ T กับความหนืดพลวัต η ของของเหลว และขนาดของละอองเรณู R

คิมจับเชิงแสง

คิมจับเชิงแสงสามารถดักจับละอองเรณูได้ ซึ่งขึ้นอยู่กับความแข็งตึงของกัก k และขนาดของละอองเรณู R โดยความแข็งตึงของกักจะดึงละอองเรณูกลับเข้าสู่พลังงานศักย์ที่ต่ำที่สุดของคิมจับเชิงแสง จึงทำให้ดักจับละอองเรณูได้

ฟังก์ชันการสูญสลาย

ฟังก์ชันการสูญสลายสามารถอธิบายสภาวะไม่สมดุลที่เกิดจากการจับอนุภาคจาก k_0 ไปยัง k_1 เมื่อเวลาผ่านไปนาน ๆ จะเห็นได้ว่าระบบไม่สามารถผันกลับได้ตามสมการของกฎข้อที่ 2 ของอุณหพลศาสตร์

ทฤษฎีบทความผันผวน

ทฤษฎีบทความผันผวนชั่วครู่ของทฤษฎีบทความผันผวน ได้ถูกยืนยันความถูกต้องด้วยการจำลองและเชื่อมโยงกับกฎข้อที่ 2 ของอุณหพลศาสตร์

5.2. ข้อเสนอแนะ

1. โปรแกรมการเคลื่อนที่แบบบราวน์สามารถเปลี่ยนพารามิเตอร์ต่าง ๆ ในสมการกระบวนการแพร่ และการดักจับเชิงแสงเพื่อดูความแตกต่างได้
2. โปรแกรมทฤษฎีบทความผันผวนสามารถเปลี่ยนพารามิเตอร์ต่าง ๆ เพื่อดูความแตกต่างได้
3. ผู้ที่สนใจสามารถนำโปรแกรมไปพัฒนาเป็นระบบ 3 มิติ ได้

บรรณานุกรม

- [1] G. G. Stokes, “On the effect of the internal friction of fluids on the motion of pendulums,” Transactions of the Cambridge Philosophical Society, **jourvol** X, **Part** II, **pages** [8]–[106], 1851.
- [2] R. Brown, “XXVII. A brief account of microscopical observations made in the months of June, July and August 1827, on the particles contained in the pollen of plants; and on the general existence of active molecules in organic and inorganic bodies,” *The philosophical magazine*, **jourvol** 4, **number** 21, **pages** 161–173, 1828. DOI: [10.1080/14786442808674769](https://doi.org/10.1080/14786442808674769).
- [3] P. Pearle, B. Collett, K. Bart, D. Bilderback, D. Newman **and** S. Samuels, “What Brown saw and you can too,” *American Journal of Physics*, **jourvol** 78, **number** 12, **pages** 1278–1289, 2010. DOI: [10.1119/1.3475685](https://doi.org/10.1119/1.3475685). arXiv: [1008.0039](https://arxiv.org/abs/1008.0039).
- [4] A. Einstein, “Über die von der molekularkinetischen Theorie der Wärme geforderte Bewegung von in ruhenden Flüssigkeiten suspendierten Teilchen (German),” **jourvol** 322, **number** 8, **pages** 549–560, 1905. DOI: [10.1002/andp.19053220806](https://doi.org/10.1002/andp.19053220806).
- [5] A. Einstein, *Investigations on the Theory of the Brownian Movement* (Dover Books on Physics Series). Dover Publications, 1956. URL: https://books.google.co.th/books?id=AOIVupH_hboC.
- [6] P. Mörters **and** Y. Peres, *Brownian Motion*. Cambridge University Press, 2010, **volume** 30. DOI: [10.1017/CBO9780511750489](https://doi.org/10.1017/CBO9780511750489). URL: <https://www.mi.uni-koeln.de/~moerters/book/book.pdf>.
- [7] P. Langevin, “Sur la théorie du mouvement brownien (French),” 1908.

- [8] D. S. Lemons **and** A. Gythiel, “Paul Langevin’s 1908 paper “On the Theory of Brownian Motion”,” *jourvol* 65, **pages** 1079–1081, **january** 1997. DOI: [10.1119/1.18725](https://doi.org/10.1119/1.18725). URL: https://www.researchgate.net/publication/252180344_Paul_Langevin's_1908_paper_On_the_Theory_of_Brownian_Motion’.
- [9] A. Medved, R. Davis **and** P. A. Vasquez, “Understanding Fluid Dynamics from Langevin and Fokker-Planck Equations,” 2020. DOI: [10.3390/fluids5010040](https://doi.org/10.3390/fluids5010040).
- [10] X. Shang **and** M. Kröger, “Time Correlation Functions of Equilibrium and Nonequilibrium Langevin Dynamics: Derivations and Numerics Using Random Numbers,” *SIAM Review*, *jourvol* 62, **number** 4, **pages** 901–935, 2020. DOI: [10.1137/19M1255471](https://doi.org/10.1137/19M1255471). arXiv: [1810.12650](https://arxiv.org/abs/1810.12650).
- [11] B. M. Schäfer, *Statistical Physics Linking the Microscopic with the Macroscopic World*. Heidelberg University Publishing, 2022. DOI: [10.17885/heup.1058](https://doi.org/10.17885/heup.1058).
- [12] P. Villegas, S. Di Santo, R. Burioni **and** M. A. Muñoz, “Time-series thresholding and the definition of avalanche size,” *Physical Review E*, *jourvol* 100, **number** 1, **pages** 012133, 2019. DOI: [10.1103/PhysRevE.100.012133](https://doi.org/10.1103/PhysRevE.100.012133). arXiv: [1902.10465](https://arxiv.org/abs/1902.10465). URL: <https://digibug.ugr.es/handle/10481/70630>.
- [13] A. Ashkin, “Acceleration and Trapping of Particles by Radiation Pressure,” *Phys. Rev. Lett.*, *jourvol* 24, **pages** 156–159, 4 **january** 1970. DOI: [10.1103/PhysRevLett.24.156](https://doi.org/10.1103/PhysRevLett.24.156).
- [14] Q. Yu **and** B. M. Hennelly, “A new spin on Ashkin’s laser trapping forces in the ray optics regime,” *Optics and Lasers in Engineering*, *jourvol* 170, **pages** 107764, 2023, ISSN: 0143-8166. DOI: [10.1016/j.optlaseng.2023.107764](https://doi.org/10.1016/j.optlaseng.2023.107764).

- [15] A. Ashkin, “Forces of a single-beam gradient laser trap on a dielectric sphere in the ray optics regime,” *Biophysical journal*, **jourvol** 61, **number** 2, **pages** 569–582, 1992. DOI: [10.1016/S0006-3495\(92\)81860-X](https://doi.org/10.1016/S0006-3495(92)81860-X).
- [16] Y. Harada **and** T. Asakura, “Radiation forces on a dielectric sphere in the Rayleigh scattering regime,” *Optics communications*, **jourvol** 124, **number** 5-6, **pages** 529–541, 1996. DOI: [10.1016/0030-4018\(95\)00753-9](https://doi.org/10.1016/0030-4018(95)00753-9). URL: https://www.researchgate.net/publication/222382598_Radiation_forces_on_a_dielectric_sphere_in_the_Rayleigh_scattering_regime.
- [17] C. Bustamante, W. Cheng **and** Y. X. Mejia, “Revisiting the Central Dogma One Molecule at a Time,” 2011. DOI: [10.1016/j.cell.2011.01.033](https://doi.org/10.1016/j.cell.2011.01.033).
- [18] NCERT, *Physics Textbook Part I for Class XI*. National Council of Educational Research and Training, 2014. URL: <https://ncert.nic.in/textbook.php?keph1=0-7>.
- [19] J. K. Blitzstein **and** J. Hwang, *Introduction to probability*. Chapman **and** Hall/CRC, 2019. DOI: [10.1201/9780429428357](https://doi.org/10.1201/9780429428357). URL: <https://probabilitybook.net/>.
- [20] D. J. Evans, E. G. D. Cohen **and** G. P. Morriss, “Probability of second law violations in shearing steady states,” *Physical review letters*, **jourvol** 71, **number** 15, **pages** 2401, 1993. DOI: [10.1103/PhysRevLett.71.3616](https://doi.org/10.1103/PhysRevLett.71.3616). URL: https://www.researchgate.net/publication/13235066_Probability_of_Second_Law_Violations_in_Shearing_Steady_States.
- [21] D. J. Searles **and** D. J. Evans, “The fluctuation theorem and Green–Kubo relations,” *The Journal of Chemical Physics*, **jourvol** 112, **number** 22, **pages** 9727–9735, 2000. DOI: [10.1063/1.481610](https://doi.org/10.1063/1.481610). arXiv: [cond-mat/9902021](https://arxiv.org/abs/cond-mat/9902021). URL: <https://hdl.handle.net/1885/40715>.

- [22] J. C. Reid, D. M. Carberry, G. M. Wang, E. M. Sevick **and** D. J. Evans, “Reversibility in nonequilibrium trajectories of an optically trapped particle,” *Phys. Rev. E*, **jourvol** 70, **pages** 016111, 1 **july** 2004. DOI: [10.1103/PhysRevE.70.016111](https://doi.org/10.1103/PhysRevE.70.016111). URL: https://www.researchgate.net/publication/8388181_Reversibility_in_nonequilibrium_trajectories_of_an_optically_trapped_particle.
- [23] D. M. Carberry, J. C. Reid, G. M. Wang, E. M. Sevick, D. J. Searles **and** D. J. Evans, “Fluctuations and Irreversibility: An Experimental Demonstration of a Second-Law-Like Theorem Using a Colloidal Particle Held in an Optical Trap,” **april** 2004. DOI: [10.1103/PhysRevLett.92.140601](https://doi.org/10.1103/PhysRevLett.92.140601). URL: <https://hdl.handle.net/10440/865>.
- [24] C. F. Petersen, “An Investigation Into the Significance of Dissipation in Statistical Mechanics,” PhD thesis, Australian National University, 2016. DOI: [10.25911/5d76378a139a1](https://doi.org/10.25911/5d76378a139a1). URL: <https://hdl.handle.net/1885/110514>.
- [25] D. J. Evans **and** D. J. Searles, “The Fluctuation Theorem,” *Advances in Physics*, **jourvol** 51, **number** 7, **pages** 1529–1585, 2002. DOI: [10.1080/00018730210155133](https://doi.org/10.1080/00018730210155133). URL: https://www.researchgate.net/publication/232969702_The_Fluctuation_Theorem.
- [26] D. J. Searles **and** D. J. Evans, “Ensemble dependence of the transient fluctuation theorem,” *The Journal of Chemical Physics*, **jourvol** 113, **number** 9, **pages** 3503–3509, 2000. DOI: [10.1063/1.1287424](https://doi.org/10.1063/1.1287424). arXiv: [cond-mat/9906002](https://arxiv.org/abs/cond-mat/9906002). URL: <https://hdl.handle.net/1885/15940>.
- [27] D. Carberry, “Optical Tweezers: Experimental Demonstrations of the Fluctuation Theorem,” PhD thesis, Australian National University, 2005. DOI: [10.25911/5d7a2a665845c](https://doi.org/10.25911/5d7a2a665845c). URL: <https://hdl.handle.net/1885/46235>.
- [28] D. J. Evans, D. J. Searles **and** S. R. Williams, “Dissipation and the relaxation to equilibrium,” **jourvol** 2009, **number** 07, 2009. DOI: [10.1088/1742-5468/2009/07/P07029](https://doi.org/10.1088/1742-5468/2009/07/P07029). arXiv: [0811.2248](https://arxiv.org/abs/0811.2248).

- [29] S. R. Williams, D. J. Searles **and** D. J. Evans, “Independence of the transient fluctuation theorem to thermostating details,” *Phys. Rev. E*, **jourvol** 70, **pages** 066113, 6 **december** 2004. DOI: [10.1103/PhysRevE.70.066113](https://doi.org/10.1103/PhysRevE.70.066113). URL: https://www.researchgate.net/publication/8036131_Independence_of_the_transient_fluctuation_theorem_to_thermostating_details.
- [30] D. J. Evans, D. J. Searles **and** S. R. Williams, “On the fluctuation theorem for the dissipation function and its connection with response theory,” *jourvol* 128, **number** 1, **pages** 014504, 2008. DOI: [10.1063/1.2812241](https://doi.org/10.1063/1.2812241). URL: <https://hdl.handle.net/1885/16698>.
- [31] D. M. Carberry, S. R. Williams, G. M. Wang, E. M. Sevick **and** D. J. Evans, “The Kawasaki identity and the Fluctuation Theorem,” *jourvol* 121, **number** 17, **pages** 8179–8182, 2004. DOI: [10.1063/1.1802211](https://doi.org/10.1063/1.1802211). URL: <https://hdl.handle.net/1885/15803>.
- [32] G. Volpe **and** G. Volpe, “Simulation of a Brownian particle in an optical trap,” *American Journal of Physics*, **jourvol** 81, **number** 3, **pages** 224–230, **march** 2013. DOI: [10.1119/1.4772632](https://doi.org/10.1119/1.4772632). URL: https://www.researchgate.net/publication/253329349_AJP000224.
- [33] G. Volpe **and** G. Volpe, “Numerical Simulation of Optically Trapped Particles,” *ETOP 2013 Proceedings*, **pages** EWP38, 2013. URL: <https://opg.optica.org/abstract.cfm?URI=ETOP-2013-EWP38>.
- [34] R. Weast, *CRC Handbook of Chemistry and Physics (1st Student Edition)*. CRC Press, 1988. URL: <https://archive.org/details/crchandbookofche01edunse>.

แหล่งข้อมูลออนไลน์

- [35] J. Southard. (2006). “Introduction to Fluid Motions, Sediment Transport, and Current-Generated Sedimentary Structures : CHAPTER 3 FLOW PAST A SPHERE II: STOKES’ LAW, THE BERNOULLI EQUATION, TURBULENCE, BOUNDARY LAYERS, FLOW SEPARATION,” Massachusetts

- Institute of Technology. URL: https://ocw.mit.edu/courses/12-090-introduction-to-fluid-motions-sediment-transport-and-current-generated-sedimentary-structures-fall-2006/7841d9b1681d6748fa2f5cbc6d6f1cb2_ch3.pdf. (accessed: 13.05.2025).
- [36] S. Mahajan. (2008). “The Art of Approximation in Science and Engineering : 8.3 Drag,” Massachusetts Institute of Technology. URL: https://ocw.mit.edu/courses/6-055j-the-art-of-approximation-in-science-and-engineering-spring-2008/39272d9d783907d9cfa779a1dedb31b9_apr30.pdf. (accessed: 13.05.2025).
- [37] P. Pearle, K. Bart, D. Bilderback, B. Collett, D. Newman **and** S. Samuels. (2010). “What Brown Saw and You Can Too : V. Microscopy : A. Growing Clarkia,” Hamilton College. URL: <https://physerver.hamilton.edu/research/brownian/Microscopy.html>. (accessed: 18.08.2025).
- [38] MikeRun. (2023). “File:Temperature-dependent-diffusion.jpg - Wikimedia Commons.” URL: <https://commons.wikimedia.org/wiki/File:Temperature-dependent-diffusion.jpg>. (accessed: 19.08.2025).
- [39] S. Lalley. (2013). “Stochastic Processes II : BROWNIAN MOTION,” The University of Chicago. URL: <https://galton.uchicago.edu/~lalley/Courses/313/BrownianMotionCurrent.pdf>. (accessed: 20.05.2025).
- [40] “Quick-Start Guide - aleatory 1.1.1 documentation.” URL: <https://aleatory.readthedocs.io/en/latest/general.html>. (accessed: 19.08.2025).
- [41] D. Sept. (2008). “Stochastic Dynamics, Brownian Dynamics and Diffusion,” Washington University in St. Louis. URL: <https://dasher.wustl.edu/bio5476/reading/stochastic.pdf>. (accessed: 15.03.2025).

ภาคผนวก

ภาคผนวก ก

ซอร์สโค้ด
(Source Code)

ก.1. ซอร์สโค้ดภายนอก

Python code 1 : math_external

```

1 import math
2 import numpy as np
3
4 π = math.pi
5 pi = math.pi
6
7 def sqrt(s):
8     return math.sqrt(s)
9
10 class vec2:
11     def __init__(self,x = 0,y = 0):
12         self.x = []
13         self.y = []
14         self.x = x
15         self.y = y
16     def __add__(self, other):
17         return vec2(self.x + other.x,self.y + other.y)
18     def __sub__(self, other):
19         if type(other) == int or type(other) == float:
20             return vec2(self.x - other,self.y - other)
21         return vec2(self.x - other.x,self.y - other.y)
22     def __rsub__(self, other):
23         if type(other) == int or type(other) == float:
24             return vec2(other-self.x,other-self.y)
25         return vec2(other.x-self.x,other.y-self.y)
26     def __mul__(self, other):
27         if type(other) == int or type(other) == float:
28             return vec2(self.x * other,self.y * other)
29         return vec2(self.x * other.x,self.y * other.y)
30     def __truediv__(self, other):
31         if type(other) == int or type(other) == float:
32             return vec2(self.x / other,self.y / other)
33         return vec2(self.x / other.x,self.y / other.y)
34     def __rtruediv__(self, other):
35         if type(other) == int or type(other) == float:
36             return vec2(other / self.x,other / self.y)
37         return vec2(other.x / self.x,other.y / self.y)
38     def __rmul__(self, other):
39         if type(other) == int or type(other) == float:
40             return vec2(self.x * other,self.y * other)
41         return vec2(self.x * other.x,self.y * other.y)
42     def __neg__(self):
43         return vec2(-self.x,-self.y)
44     def __pow__(self, other):
45         return vec2(pow(self.x,other),pow(self.y,other))
46     def __eq__(self, other):
47         return (self.x, self.y) == (other.x, other.y)
48     def __str__(self):
49         return str(self.x) + " , " + str(self.y)

```

ก.2. การเคลื่อนที่แบบบราวน์ 2 มิติ

ก.2.1. จินตทัศน์

Python code 2 : clip_line

```

1  # Liang-Barsky Line-Clip Algorithm
2  # https://dl.acm.org/doi/pdf/10.1145/357332.357333
3
4  from math_external import *
5
6  x_min, y_min = -1.0, -1.0
7  x_max, y_max = 1.0, 1.0
8
9  t0 = 0.0
10 t1 = 1.0
11
12 def clipt(p,q):
13     global t0,t1
14     accept = True
15     if p < 0:
16         t = q/p
17         if t > t1:
18             accept = False
19         elif t > t0:
20             t0 = t
21     elif p > 0:
22         t = q/p
23         if t < t0:
24             accept = False
25         elif t < t1:
26             t1 = t
27     else: #p = 0
28         if q < 0:
29             accept = False
30     return accept
31
32 def clip2d(x0, y0, x1, y1):
33     global t0,t1
34     t0 = 0.0
35     t1 = 1.0
36     Δx = x1 - x0
37
38     if clipt(-Δx, x0 - x_min):
39         if clipt(Δx, x_max - x0):
40             Δy = y1 - y0
41             if clipt(-Δy, y0 - y_min):
42                 if clipt(Δy, y_max - y0):
43                     if t1 < 1:
44                         x1 = x0 + t1*Δx
45                         y1 = y0 + t1*Δy
46                     if t0 > 0:
47                         x0 = x0 + t0*Δx
48                         y0 = y0 + t0*Δy
49     return [vec2(x0,y0),vec2(x1,y1)]

```

Python code 3 : จินตทัศน์ของการเคลื่อนที่แบบบราวน์ 2 มิติ

```

1 import io
2 import sys
3 sys.dont_write_bytecode = True
4 import copy
5 import numpy as np
6 from OpenGL import GL, GLU
7 import glfw as GLFW
8
9 from imgui_bundle import imgui
10 from imgui_bundle.python_backends.glfw_backend import GlfwRenderer
11
12 ContourID = -1
13 ColorBarID = -1
14 cbw1 = 0
15 cbh1 = 0
16
17 from pathlib import Path
18 path = str(Path(__file__).parent.absolute())+"/"
19 prev_path = path+"../"
20
21 FPS = 100
22 Δt = 1.0 / FPS
23 stop = True
24
25 from math_external import *
26
27 denom = 2.5e+9
28
29 width = 900
30 width_downscale = width/denom
31 height = 900
32 height_downscale = height/denom
33
34 import clip_line as cl
35 cl.x_min = -width_downscale
36 cl.y_min = -height_downscale
37 cl.x_max = width_downscale
38 cl.y_max = height_downscale
39
40 position = vec2(0,0)
41
42 position_upscale = vec2(0,0)
43 data_position = []
44 data_position.append(vec2(position.x,position.y))
45 data_line = []
46
47 pollen = True
48 mark = False
49 walk = True
50
51 η = 1.137*1e-3 # viscosity
52 R = 10*1e-6 # radius
53 R_downscale = R*1e-3 # radius
54 kB = 1.380649e-23 # boltzmann constant
55 T = 288 # temperature
56
57 γ = 6*π*η*R

```

```

58 k = vec2(1.22*1e-6,1.22*1e-6)
59 F_ext = -k*position
60
61 def reset():
62     global position,data_position,data_line
63     position = vec2(0,0)
64     data_position = []
65     data_line = []
66
67     global k,η,R,R_downscale,kB,T
68     η = 1.137*1e-3 # viscosity
69     R = 10*1e-6 # radius
70     R_downscale = R*1e-3 # radius
71     kB = 1.380649e-23 # boltzmann constant
72     T = 288 # temperature
73     k = vec2(1.22*1e-6,1.22*1e-6)
74
75 def reset_walk():
76     global position,data_position,data_line
77     position = vec2(0,0)
78     data_position = []
79     data_line = []
80
81 def random_position():
82     global position,data_position
83
84     μx = 0
85     σx = np.sqrt(kB*T/k.x)
86     sx = np.random.normal(μx, σx, 1)
87
88     μy = 0
89     σy = np.sqrt(kB*T/k.y)
90     sy = np.random.normal(μy, σy, 1)
91
92     position = vec2(sx[0],sy[0])
93
94     data_position.append(vec2(position.x,position.y))
95
96 def step(dt):
97     global position,data_position,data_line,D,η,γ,F_ext
98     old_position = copy.deepcopy(position)
99
100     γ = 6*π*η*R # macroscopic hydrodynamics
101     D = (kB*T) # diffusion coefficient
102
103     μx = 0
104     σx = 1
105     sx = np.random.normal(μx, σx, 1)
106     μy = 0
107     σy = 1
108     sy = np.random.normal(μy, σy, 1)
109     random_walk = vec2(sx[0],sy[0])
110
111     F_ext = -k*position
112     position = position + (F_ext/γ)*dt + sqrt((2*kB*T*dt)/γ)*random_walk
113
114     backup_position = copy.deepcopy(position)
115     position_swap = copy.deepcopy(position)
116

```

```

117     if position.x > width_downscale:
118         position_swap.x -= width_downscale*2*2
119     elif position.x < -width_downscale:
120         position_swap.x += width_downscale*2*2
121
122     if position.y > height_downscale:
123         position_swap.y -= height_downscale*2*2
124     elif position.y < -height_downscale:
125         position_swap.y += height_downscale*2*2
126
127     if position.x > width_downscale:
128         position.x -= width_downscale*2
129     elif position.x < -width_downscale:
130         position.x += width_downscale*2
131
132     if position.y > height_downscale:
133         position.y -= height_downscale*2
134     elif position.y < -height_downscale:
135         position.y += height_downscale*2
136
137     if backup_position != position:
138         data = cl.clip2d(old_position.x,old_position.y,backup_position.x,backup_posi
↪ tion.y)
139         if data:
140             data_line.append([data[0],data[1]])
141             data_position.append(data[0])
142
143         data = cl.clip2d(position_swap.x,position_swap.y,position.x,position.y)
144         if data:
145             data_line.append([data[0],data[1]])
146             data_position.append(data[1])
147     else:
148         data_position.append(vec2(position.x,position.y))
149         data_line.append([vec2(old_position.x,old_position.y),vec2(position.x,positi
↪ on.y)])
150
151 def set_view():
152     GL.glMatrixMode(GL.GL_PROJECTION)
153     GL.glLoadIdentity()
154
155     GLU.gluOrtho2D(-width_downscale,width_downscale,-height_downscale,height_downsca
↪ le)
156
157     GL.glMatrixMode(GL.GL_MODELVIEW)
158     GL.glLoadIdentity()
159
160 from draw import *
161
162 def render():
163     GL.glClear(GL.GL_COLOR_BUFFER_BIT|GL.GL_DEPTH_BUFFER_BIT)
164     GL.glClearColor(0,0,0,1)
165
166     set_view()
167
168     GL.glDisable(GL.GL_DEPTH_TEST)
169     GL.glColor4f(255,255,255,255)
170     GL.glEnable(GL.GL_TEXTURE_2D)
171     GL.glBindTexture(GL.GL_TEXTURE_2D, ContourID)

```

```

172     GL.glPushMatrix()
173     draw_filled_aabb_min_max(-width_downscale,width_downscale,-height_downscale,height_downscale)
174     GL.glPopMatrix()
175     GL.glBindTexture(GL.GL_TEXTURE_2D, 0)
176     GL.glDisable(GL.GL_TEXTURE_2D)
177     GL.glEnable(GL.GL_DEPTH_TEST)
178
179     if pollen:
180         GL.glClipPlane(GL.GL_CLIP_PLANE0, (1.0, 0.0, 0.0, width_downscale))
181         GL.glClipPlane(GL.GL_CLIP_PLANE1, (-1.0, 0.0, 0.0, width_downscale))
182         GL.glClipPlane(GL.GL_CLIP_PLANE2, (0.0, 1.0, 0.0, height_downscale))
183         GL.glClipPlane(GL.GL_CLIP_PLANE3, (0.0, -1.0, 0.0, height_downscale))
184
185         GL.glEnable(GL.GL_CLIP_PLANE0)
186         GL.glEnable(GL.GL_CLIP_PLANE1)
187         GL.glEnable(GL.GL_CLIP_PLANE2)
188         GL.glEnable(GL.GL_CLIP_PLANE3)
189
190         GL.glPushMatrix()
191         GL.glColor3ub(255,0,0)
192         GL.glTranslatef(position.x,position.y,0)
193         draw_filled_circle(0,0,R_downscale)
194         GL.glPopMatrix()
195
196         draw_boundary_condition(R_downscale,position,width_downscale,height_downscale)
197
198         GL.glDisable(GL.GL_CLIP_PLANE0)
199         GL.glDisable(GL.GL_CLIP_PLANE1)
200         GL.glDisable(GL.GL_CLIP_PLANE2)
201         GL.glDisable(GL.GL_CLIP_PLANE3)
202
203     if mark:
204         for p in data_position:
205             GL.glPushMatrix()
206             GL.glColor3ub(255,165,0)
207             GL.glBegin(GL.GL_POINTS)
208             GL.glVertex2f(p.x,p.y)
209             GL.glEnd()
210             GL.glPopMatrix()
211
212     if walk:
213         GL.glPushMatrix()
214         data_line_size = len(data_line)
215         count = 0
216         GL.glColor3ub(0,255,0)
217         GL.glBegin(GL.GL_LINES)
218         while count < data_line_size:
219             p1 = data_line[count][0]
220             p2 = data_line[count][1]
221             GL.glVertex2f(p1.x,p1.y)
222             GL.glVertex2f(p2.x,p2.y)
223             count += 1
224         GL.glEnd()
225         GL.glPopMatrix()
226
227     import matplotlib.pyplot as plt
228     from matplotlib_scalebar.scalebar import ScaleBar

```

```

229 def gen_contour_image():
230     fig = plt.figure(frameon=False)
231     ax = fig.add_axes([0, 0, 1, 1])
232     ax.set_facecolor("black")
233
234     x = np.linspace(-width_downscale, width_downscale, 500)
235     y = np.linspace(-height_downscale, height_downscale, 500)
236
237     X, Y = np.meshgrid(x, y)
238     U = 0.5 * (k.x * X**2 + k.y * Y**2)
239
240     cp = plt.contourf(X, Y, U, cmap="viridis")
241
242     ax.add_artist(ScaleBar(1,location=4,color='lime',box_color='black',box_alpha=0.5))
243     ax.add_artist(ScaleBar(1*1e+3,location=1,color='red',box_color='black',box_alpha=
↵ =0.5))
244
245     img_buf = io.BytesIO()
246     plt.savefig(img_buf, format='png', dpi=150)
247     return img_buf
248
249 import matplotlib.cm as cm
250 import matplotlib.colors as colors
251
252 from matplotlib.ticker import ScalarFormatter
253
254 class UnitFormatter(ScalarFormatter):
255     def __init__(self, unit="m", **kwargs):
256         self.unit = unit
257         super().__init__(**kwargs)
258
259     def get_offset(self):
260         offset = super().get_offset()
261         if offset:
262             return offset + f' {self.unit}'
263         return ''
264
265 formatter2 = UnitFormatter(unit='J', useOffset=True, useMathText=True)
266
267 def gen_potential_colorbar_image():
268     fig = plt.figure(figsize=(2, 6))
269     ax = fig.add_axes([0.4, 0.1, 0.2, 0.8])
270     ax.set_facecolor("black")
271
272     x = np.linspace(-width_downscale, width_downscale, 500)
273     y = np.linspace(-height_downscale, height_downscale, 500)
274
275     X, Y = np.meshgrid(x, y)
276     U = 0.5 * (k.x * X**2 + k.y * Y**2)
277
278     vmin = np.min(U)
279     vmax = np.max(U)
280     norm = colors.Normalize(vmin=vmin, vmax=vmax)
281
282     sm = cm.ScalarMappable(cmap="viridis", norm=norm)
283
284     cbar = plt.colorbar(sm, cax=ax)
285

```

```

286     cbar.ax.yaxis.set_major_formatter(formatter2)
287
288     img_buf = io.BytesIO()
289     plt.savefig(img_buf, format='png', dpi=150)
290     return img_buf
291
292 from PIL import Image
293 def load_image_to_texture(filename):
294     img = Image.open(filename)
295     img_data = img.tobytes("raw", "RGBA", 0, -1)
296     img_width = img.size[0]
297     img_height = img.size[1]
298
299     texture_id = GL.glGenTextures(1)
300     GL.glBindTexture(GL.GL_TEXTURE_2D, texture_id)
301     GL.glPixelStorei(GL.GL_UNPACK_ALIGNMENT, 1)
302     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_WRAP_S, GL.GL_CLAMP)
303     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_WRAP_T, GL.GL_CLAMP)
304     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_MAG_FILTER, GL.GL_LINEAR)
305     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_MIN_FILTER, GL.GL_LINEAR)
306     GL.glTexImage2D(GL.GL_TEXTURE_2D, 0, GL.GL_RGBA, img_width, img_height, 0,
307     ↪ GL.GL_RGBA, GL.GL_UNSIGNED_BYTE, img_data)
308     GL.glBindTexture(GL.GL_TEXTURE_2D, 0)
309     return texture_id, img_width, img_height
310
311 def gen_contour():
312     img = Image.open(gen_contour_image())
313     img_data = img.tobytes("raw", "RGBA", 0, -1)
314     img_width = img.size[0]
315     img_height = img.size[1]
316
317     texture_id = GL.glGenTextures(1)
318     GL.glBindTexture(GL.GL_TEXTURE_2D, texture_id)
319     GL.glPixelStorei(GL.GL_UNPACK_ALIGNMENT, 1)
320     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_WRAP_S, GL.GL_CLAMP)
321     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_WRAP_T, GL.GL_CLAMP)
322     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_MAG_FILTER, GL.GL_LINEAR)
323     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_MIN_FILTER, GL.GL_LINEAR)
324     GL.glTexImage2D(GL.GL_TEXTURE_2D, 0, GL.GL_RGBA, img_width, img_height, 0,
325     ↪ GL.GL_RGBA, GL.GL_UNSIGNED_BYTE, img_data)
326     GL.glBindTexture(GL.GL_TEXTURE_2D, 0)
327     return texture_id
328
329 def gen_potential_colorbar():
330     img = Image.open(gen_potential_colorbar_image())
331     img_data = img.tobytes("raw", "RGBA", 0, -1)
332     img_width = img.size[0]
333     img_height = img.size[1]
334
335     texture_id = GL.glGenTextures(1)
336     GL.glBindTexture(GL.GL_TEXTURE_2D, texture_id)
337     GL.glPixelStorei(GL.GL_UNPACK_ALIGNMENT, 1)
338     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_WRAP_S, GL.GL_CLAMP)
339     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_WRAP_T, GL.GL_CLAMP)
340     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_MAG_FILTER, GL.GL_LINEAR)
341     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_MIN_FILTER, GL.GL_LINEAR)
342     GL.glTexImage2D(GL.GL_TEXTURE_2D, 0, GL.GL_RGBA, img_width, img_height, 0,
343     ↪ GL.GL_RGBA, GL.GL_UNSIGNED_BYTE, img_data)
344     GL.glBindTexture(GL.GL_TEXTURE_2D, 0)

```



```

342     return texture_id,img_width,img_height
343
344 def regen_contour():
345     global ContourID
346     img = Image.open(gen_contour_image())
347     img_data = img.tobytes("raw", "RGBA", 0, -1)
348     img_width = img.size[0]
349     img_height = img.size[1]
350
351     GL.glBindTexture(GL.GL_TEXTURE_2D, ContourID)
352     GL.glPixelStorei(GL.GL_UNPACK_ALIGNMENT, 1)
353     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_WRAP_S, GL.GL_CLAMP)
354     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_WRAP_T, GL.GL_CLAMP)
355     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_MAG_FILTER, GL.GL_LINEAR)
356     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_MIN_FILTER, GL.GL_LINEAR)
357     GL.glTexImage2D(GL.GL_TEXTURE_2D, 0, GL.GL_RGBA, img_width, img_height, 0,
358     ↪ GL.GL_RGBA, GL.GL_UNSIGNED_BYTE, img_data)
359     GL.glBindTexture(GL.GL_TEXTURE_2D, 0)
360
361     global ColorBarID
362     img = Image.open(gen_potential_colorbar_image())
363     img_data = img.tobytes("raw", "RGBA", 0, -1)
364     img_width = img.size[0]
365     img_height = img.size[1]
366
367     GL.glBindTexture(GL.GL_TEXTURE_2D, ColorBarID)
368     GL.glPixelStorei(GL.GL_UNPACK_ALIGNMENT, 1)
369     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_WRAP_S, GL.GL_CLAMP)
370     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_WRAP_T, GL.GL_CLAMP)
371     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_MAG_FILTER, GL.GL_LINEAR)
372     GL.glTexParameterf(GL.GL_TEXTURE_2D, GL.GL_TEXTURE_MIN_FILTER, GL.GL_LINEAR)
373     GL.glTexImage2D(GL.GL_TEXTURE_2D, 0, GL.GL_RGBA, img_width, img_height, 0,
374     ↪ GL.GL_RGBA, GL.GL_UNSIGNED_BYTE, img_data)
375     GL.glBindTexture(GL.GL_TEXTURE_2D, 0)
376
377 def init_opengl():
378     global ContourID,ColorBarID,cbw1,cbh1
379     ContourID = gen_contour()
380     ColorBarID,cbw1,cbh1 = gen_potential_colorbar()
381
382     GL.glPointSize(3.0)
383     GL.glEnable(GL.GL_DEPTH_TEST)
384
385 def main():
386     reset()
387     random_position()
388
389     if not GLFW.init():
390         print("could not initialize GLFW")
391         sys.exit(1)
392
393     GLFW.window_hint(GLFW.RESIZABLE,GLFW.FALSE)
394
395     window = GLFW.create_window(int(width), int(height), "PHY4906 : Brownian Motion
396     ↪ 2D", None, None)
397     GLFW.make_context_current(window)
398
399     monitor = GLFW.get_primary_monitor()
400     pos = GLFW.get_monitor_pos(monitor)

```

```

398 size = GLFW.get_window_size(window)
399 mode = GLFW.get_video_mode(monitor)
400 GLFW.set_window_pos(
401     window,
402     int(pos[0] + (mode.size.width - size[0]) / 2),
403     int(pos[1] + (mode.size.height - size[1]) / 2))
404
405 if not window:
406     GLFW.terminate()
407     print("Could not initialize Window")
408     sys.exit(1)
409
410 init_opengl()
411
412 imgui.create_context()
413 io = imgui.get_io()
414 impl = GlfwRenderer(window)
415 #io.config_debug_ini_settings = False
416 #io.want_save_ini_settings = False
417 font_atlas = io.fonts
418
419 font_atlas.add_font_default()
420 new_font = font_atlas.add_font_from_file_ttf(
421     filename=prev_path+"font/ProggyClean.ttf",
422     size_pixels=13,
423 )
424
425 font_config = imgui.ImFontConfig()
426 font_config.merge_mode = True
427
428 glyph_range = []
429 glyph_range.append(0x0080)
430 glyph_range.append(0x00FF)
431 glyph_range.append(0x0100)
432 glyph_range.append(0x017F)
433 glyph_range.append(0x2070)
434 glyph_range.append(0x209F)
435 glyph_range.append(0x2200)
436 glyph_range.append(0x22FF)
437
438 glyph_range.append(0)
439 new_font = font_atlas.add_font_from_file_ttf(
440     filename=prev_path+"font/unifont-16.0.01.otf",
441     size_pixels=16,
442     font_cfg=font_config,
443 )
444
445 font_config = imgui.ImFontConfig()
446 font_config.merge_mode = True
447
448 glyph_range = []
449 glyph_range.append(0x0001D44E)
450 glyph_range.append(0x0001D44F)
451
452 glyph_range.append(0)
453 new_font = font_atlas.add_font_from_file_ttf(
454     filename=prev_path+"font/FiraMath-Regular.otf",
455     size_pixels=18,
456     font_cfg=font_config,

```

```

457 )
458
459 time = 0.0
460 accumulator = 0.0
461 current_time = GLFW.get_time()
462
463 global FPS, Δt, stop
464 FPS_list = ["5", "10", "15", "30", "60", "100", "120", "144", "165", "240", "360"]
465 FPS_selected = 5
466 FPS = int(FPS_list[FPS_selected])
467 Δt = 1.0 / FPS
468 while not GLFW.window_should_close(window):
469     GLFW.poll_events()
470     impl.process_inputs()
471
472     new_time = GLFW.get_time()
473     frame_time = new_time - current_time
474     current_time = new_time
475     if stop == False:
476         accumulator += frame_time
477         while (accumulator >= Δt):
478             step(Δt)
479             accumulator -= Δt
480             time += Δt
481
482     imgui.new_frame()
483
484     imgui.push_font(new_font, 13.0)
485     imgui.begin("Parameter")
486
487     global R, R_downscale, T, η, k, pollen, mark, walk
488
489     imgui.text("Radius (R)")
490     changed, R = imgui.slider_float("×106 m", R*1e+6, 10, 100)
491     R = R*1e-6
492     R_downscale = R*1e-3
493     imgui.text("Temperature (T)")
494     #changed, T = imgui.slider_float("K", T, 0.0, 500.0)
495     changed, T = imgui.input_float("K", T, 0.0, 500.0)
496     imgui.text("Viscosity (η)")
497     #changed, η = imgui.slider_float("×103 Pa·s", η*1e+3, 0.001, 5)
498     changed, η = imgui.input_float("×103 Pa·s", η*1e+3, 0.001, 50)
499     η = η*1e-3
500     imgui.text("Trap Stiffness (k)")
501     changed, k.x = imgui.slider_float("×106 N/m", k.x*1e+6, 0, 4.0)
502     changed, k.y = imgui.slider_float("×106 N/m", k.y*1e+6, 0, 4.0)
503     k.x = k.x*1e-6
504     k.y = k.y*1e-6
505
506     changed, pollen = imgui.checkbox("show pollen", pollen)
507     changed, mark = imgui.checkbox("show mark", mark)
508     changed, walk = imgui.checkbox("show walk", walk)
509
510     if imgui.button('Reset All'):
511         reset()
512         random_position()
513         regen_contour()
514         time = 0
515

```

```

516     if imgui.button('Reset Walk'):
517         reset_walk()
518         if not (k.x == 0 and k.y == 0):
519             random_position()
520             time = 0
521
522     if imgui.button('Regen Contour'):
523         regen_contour()
524
525     if stop:
526         if imgui.button('Continue'):
527             stop = not stop
528     else:
529         if imgui.button('Stop'):
530             stop = not stop
531
532     if imgui.begin_combo("FPS", FPS_list[FPS_selected]):
533         for i, item in enumerate(FPS_list):
534             is_selected = (i == FPS_selected)
535             if imgui.selectable(item, is_selected)[0]:
536                 FPS = int(FPS_list[i])
537                 Δt = 1.0 / FPS
538                 FPS_selected = i
539
540             if is_selected:
541                 imgui.set_item_default_focus()
542
543     imgui.end_combo()
544
545     imgui.text("FPS: %d, Δt: %f" % (FPS, Δt))
546     imgui.text("t: %f s" % (time))
547     imgui.text("\n")
548
549     imgui.text("F|$_\mathrm{ext}$|: (%f,%f)×1013 N" % (F_ext.x*1e+13, F_ext.y*1e+13))
550
551     imgui.end()
552     imgui.pop_font()
553
554     imgui.begin("Potential Energy (U)")
555     imgui.image(imgui.ImTextureRef(ColorBarID),imgui.ImVec2(cbw1*0.5,
556 ↪ cbh1*0.5),imgui.ImVec2(0,1),imgui.ImVec2(1,0))
557     imgui.end()
558
559     render()
560     imgui.render()
561     impl.render(imgui.get_draw_data())
562     GLFW.swap_buffers(window)
563
564     GLFW.destroy_window(window)
565     GLFW.terminate()
566     return 0
567
568 if __name__ == "__main__":
569     sys.exit(main())

```

ก.2.2. สมการกระบวนการแพร่

Python code 4 : สมการกระบวนการแพร่

```

1  import sys
2  sys.dont_write_bytecode = True
3  import numpy as np
4  import matplotlib.pyplot as plt
5
6  from math_external import *
7
8  FPS = 100
9  Δt = 1.0 / FPS
10 N = 10000
11
12 kB = 1.380649e-23
13 π = np.pi
14 R = 10*1e-6
15
16 T = 288.15
17 η = 1.139*1e-3
18
19 γ = 6*π*η*R
20 D = (kB*T)/γ
21
22 μ, σ = 0, 1
23 w_x = np.random.normal(μ, σ, N)
24 w_y = np.random.normal(μ, σ, N)
25
26 position = vec2(0,0)
27 data_position_x = []
28 data_position_x.append(position.x)
29 data_position_y = []
30 data_position_y.append(position.y)
31
32 for i in range(N):
33     random_walk = vec2(w_x[i],w_y[i])
34     position = position + sqrt(2*D*Δt)*random_walk
35     data_position_x.append(position.x)
36     data_position_y.append(position.y)
37
38 fig1 = plt.figure(1,figsize=(8,8))
39 ax1 = fig1.gca()
40
41 from matplotlib.ticker import ScalarFormatter
42
43 class UnitFormatter(ScalarFormatter):
44     def __init__(self, unit="m", **kwargs):
45         self.unit = unit
46         super().__init__(**kwargs)
47
48     def get_offset(self):
49         offset = super().get_offset()
50         if offset:
51             return offset + f' {self.unit}'
52         return ''
53
54 formatter = UnitFormatter(unit='m', useOffset=True, useMathText=True)
55 formatter.set_scientific(True)
56 formatter.set_powerlimits((0, 10))
57

```

```

58 ax1.xaxis.set_major_formatter(formatter)
59 ax1.yaxis.set_major_formatter(formatter)
60
61 plt.xlabel('X')
62 plt.ylabel('Y')
63
64 plt.plot(data_position_x, data_position_y, color="tab:blue")
65
66 plt.plot(data_position_x[0],data_position_y[0], 'mo')
67 plt.plot(data_position_x[len(data_position_x)-1],data_position_y[len(data_position_y)
↵ )-1], 'ro')
68
69 from matplotlib_scalebar.scalebar import ScaleBar
70 ax1.add_artist(ScaleBar(1,location=4,color='tab:blue',box_color='black',box_alpha=0.
↵ 5))
71
72 from pathlib import Path
73 path = str(Path(__file__).parent.absolute())+"/"
74 plt.savefig(path+"LE-2D"+".png", dpi=200)
75
76 plt.show()

```

ก.2.3. คีมจับเชิงแสง

Python code 5 : คีมจับเชิงแสง

```

1 import sys
2 sys.dont_write_bytecode = True
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 from math_external import *
7
8 FPS = 100
9 Δt = 1.0 / FPS
10 N = 500
11
12 kB = 1.380649e-23
13 T = 288
14
15 π = np.pi
16 η = 1.137*1e-3
17
18 R = 10*1e-6
19 x_min = -1.2e-10
20 x_max = 1.2e-10
21 y_min = -1.2e-10
22 y_max = 1.2e-10
23
24 position = vec2(0,0)
25 data_position_x = []
26 data_position_x.append(position.x)
27 data_position_y = []
28 data_position_y.append(position.y)
29

```

```

30 k = vec2(1.22*1e-6,1.22*1e-6)
31
32 for i in range(N):
33     y = 6*π*η*R # macroscopic hydrodynamics
34     D = (kB*T) # diffusion coefficient
35
36     wx = np.random.normal(0, 1, 1)
37     wy = np.random.normal(0, 1, 1)
38     random_walk = vec2(wx[0],wy[0])
39
40     F_ext = -k*position
41     position = position + (F_ext/y)*Δt + sqrt(2*D*Δt)*random_walk
42     data_position_x.append(position.x)
43     data_position_y.append(position.y)
44
45 fig1 = plt.figure(1,figsize=(6,6))
46 fig1.canvas.manager.resize(600, 600)
47 ax1 = fig1.gca()
48
49 from matplotlib.ticker import ScalarFormatter
50
51 class UnitFormatter(ScalarFormatter):
52     def __init__(self, unit="m", **kwargs):
53         self.unit = unit
54         super().__init__(**kwargs)
55
56     def get_offset(self):
57         offset = super().get_offset()
58         if offset:
59             return offset + f' {self.unit}'
60         return ''
61
62 formatter = UnitFormatter(unit='m', useOffset=True, useMathText=True)
63 formatter.set_scientific(True)
64 formatter.set_powerlimits((0, 10))
65
66 ax1.xaxis.set_major_formatter(formatter)
67 ax1.yaxis.set_major_formatter(formatter)
68
69 plt.xlabel('X')
70 plt.ylabel('Y')
71
72 plt.plot(data_position_x, data_position_y, color="lime", linewidth=0.5)
73
74 plt.plot(data_position_x[0],data_position_y[0],'mo')
75 plt.plot(data_position_x[len(data_position_x)-1],data_position_y[len(data_position_y)
↵ -1],'ro')
76
77 x = np.linspace(x_min, x_max, 500)
78 y = np.linspace(y_min, y_max, 500)
79 X, Y = np.meshgrid(x, y)
80
81 U = 0.5 * (k.x * X**2 + k.y * Y**2)
82
83 cf1 = plt.contourf(X, Y, U, cmap="viridis")
84 ax1.set_aspect('equal', adjustable='box')
85 cb1 = plt.colorbar(cf1, ax=ax1, label='Potential Energy (U)')
86 formatter2 = UnitFormatter(unit='J', useOffset=True, useMathText=True)

```

```

87 cb1.ax.yaxis.set_major_formatter(formatter2)
88
89 from matplotlib_scalebar.scalebar import ScaleBar
90 ax1.add_artist(ScaleBar(1,location=4,color='lime',box_color='black',box_alpha=0.5))
91
92 ax1.set_xlim(x_min,x_max)
93 ax1.set_ylim(y_min,y_max)
94
95 from pathlib import Path
96 path = str(Path(__file__).parent.absolute())+"/"
97 plt.savefig(path+"image1"+"png", dpi=200)
98
99 fig2 = plt.figure(2,figsize=(6,6))
100 fig2.canvas.manager.resize(600, 600)
101 ax2 = fig2.gca()
102 ax2.xaxis.set_major_formatter(formatter)
103 ax2.yaxis.set_major_formatter(formatter)
104
105 plt.xlabel('X')
106 plt.ylabel('Y')
107
108 plt.plot(data_position_x, data_position_y, color="lime", linewidth=0.5)
109
110 plt.plot(data_position_x[0],data_position_y[0],'mo')
111 plt.plot(data_position_x[len(data_position_x)-1],data_position_y[len(data_position_y)
↵ -1],'ro')
112
113 x_pot = np.linspace(min(data_position_x), max(data_position_x), 500)
114 y_pot = np.linspace(min(data_position_y), max(data_position_y), 500)
115 X, Y = np.meshgrid(x_pot, y_pot)
116
117 U = 0.5 * (k.x * X**2 + k.y * Y**2)
118 cf2 = plt.contourf(X, Y, U, cmap="viridis")
119 ax2.set_aspect('equal', adjustable='box')
120 cb2 = plt.colorbar(cf2, ax=ax2, label='Potential Energy (U)')
121 cb2.ax.yaxis.set_major_formatter(formatter2)
122
123 plt.savefig(path+"image2"+"png", dpi=200)
124
125 plt.show()

```

ก.3. ทฤษฎีบทความผันผวน

Python code 6 : ทฤษฎีบทความผันผวน

```

1 import sys
2 sys.dont_write_bytecode = True
3 import copy
4 import numpy as np
5 np.set_printoptions(legacy='1.21')
6 import math
7 π = math.pi
8
9 from math_external import *
10

```



```

11 from pathlib import Path
12 path = str(Path(__file__).parent.absolute())+"/"
13
14  $\eta$  = 1.137*1e-3 # viscosity
15 R = 10*1e-6 # radius
16 kB = 1.380649e-23 # boltzmann constant
17 T = 288 # temperature
18
19 FPS = 1000
20  $\Delta t$  = 1.0 / FPS
21 num_step = 51
22 t = 500*1e-3
23 t_step = np.linspace(0, t, num_step)
24
25 k0 = vec2(1.22*1e-6, 1.22*1e-6)
26 k1 = vec2(2.9*1e-6, 2.9*1e-6)
27
28  $\gamma = 6 * \pi * \eta * R$ 
29
30 def task(N):
31      $\mu_x$  = 0
32      $\sigma_x$  = np.sqrt(kB*T/k0.x)
33      $s_x$  = np.random.normal( $\mu_x$ ,  $\sigma_x$ , 1)
34
35      $\mu_y$  = 0
36      $\sigma_y$  = np.sqrt(kB*T/k0.y)
37      $s_y$  = np.random.normal( $\mu_y$ ,  $\sigma_y$ , 1)
38
39     initial_position = vec2( $s_x$ [0],  $s_y$ [0])
40     position = copy.deepcopy(initial_position)
41
42     list_Ω = []
43
44     time = 0
45     count_step = 0
46     while True:
47         time = time +  $\Delta t$ 
48         if time > t_step[count_step]:
49             inv_kBT = (1/(kB*T))
50             Ω = (1/2)*(inv_kBT)*(k0-k1)*(position**2-initial_position**2)
51             Ω = (Ω.x+Ω.y)
52             list_Ω.append(Ω)
53             count_step += 1
54         if time > t:
55             if (len(t_step)>count_step):
56                 inv_kBT = (1/(kB*T))
57                 Ω = (1/2)*(inv_kBT)*(k0-k1)*(position**2-initial_position**2)
58                 Ω = (Ω.x+Ω.y)
59                 list_Ω.append(Ω)
60             return list_Ω
61
62      $\mu_x$  = 0
63      $\sigma_x$  = 1
64      $s_x$  = np.random.normal( $\mu_x$ ,  $\sigma_x$ , 1)
65      $\mu_y$  = 0
66      $\sigma_y$  = 1
67      $s_y$  = np.random.normal( $\mu_y$ ,  $\sigma_y$ , 1)
68     random_walk = vec2( $s_x$ [0],  $s_y$ [0])
69
70     position = position - (k1/ $\gamma$ )*position* $\Delta t$  + sqrt((2*kB*T* $\Delta t$ )/ $\gamma$ )*random_walk

```

```

70
71 import multiprocessing
72 N = 5000
73 chunksize = int(N/10)
74 num_core = multiprocessing.cpu_count()
75
76 data_Q = []
77 with multiprocessing.Pool(num_core) as executor:
78     data_Q = executor.map(task, iterable=range(N), chunksize=chunksize)
79
80 import matplotlib.pyplot as plt
81
82 data_Q_list = []
83 for i in range(len(t_step)):
84     data_Q_id = []
85     for Q in data_Q:
86         data_Q_id.append(Q[i])
87     data_Q_list.append(data_Q_id)
88
89 bin_width = 0.2
90 bins = np.arange(-10, 10 + bin_width, bin_width)
91
92 Q_freq_data = []
93 Q_edges_data = []
94 for i in range(len(t_step)):
95     freq, edges = np.histogram(data_Q_list[i], bins=bins, density=False)
96     Q_freq_data.append(freq)
97     Q_edges_data.append(edges)
98
99 i_step = np.linspace(0, 6, 6, dtype=int)
100 size_i_step = len(i_step)
101 i_step_custom = np.linspace(7, 50, 3, dtype=int)
102 i_step = np.concatenate([i_step, i_step_custom])
103 size_i_step += len(i_step_custom)
104
105 from matplotlib.ticker import FuncFormatter
106
107 fig1, ax1 = plt.subplots(num='The Dissipation Function',
108                          nrows=3, ncols=3, figsize=(12, 12))
109
110 for i in range(3):
111     for j in range(3):
112         num = j+i*3
113         str_id = "${Q_t}$ at " + str(round(t_step[i_step[num]]*1e+3, 3)) + " ms"
114         ax1[i][j].stairs(Q_freq_data[i_step[num]], Q_edges_data[i_step[num]],
115                          ↪ fill=True)
116         ax1[i][j].set_title(str_id)
117         ax1[i][j].axvline(x=0, color='black', linestyle='--', alpha=0.4, linewidth=1.0)
118         ax1[i][j].xaxis.set_major_formatter(FuncFormatter(lambda val, pos:
119                     ↪ f"${val:.1f}$"))
120         if not (i == 0 and j == 0):
121             ax1[i][j].set_xlim(-10, 10)
122             ax1[i][j].set_ylim(0, 1500)
123
124 fig1.supxlabel('${Q_t}$')
125 fig1.supylabel('#${Q_t}$')
126
127 plt.savefig(path+"1.png", dpi=200)
128

```

```

127 fig2, ax2 = plt.subplots(num='Probability of Dissipation Function',
128                           nrows=3, ncols=3, figsize=(12, 12))
129
130 data_P_Qt = []
131 for i in range(len(t_step)):
132     Δx = np.diff(Q_edges_data[i])
133     area = sum(Q_freq_data[i]*Δx)
134     P_Qt = Q_freq_data[i]/area
135     data_P_Qt.append(P_Qt)
136
137 for i in range(3):
138     for j in range(3):
139         num = j+i*3
140         str_id = "${Q_t}$ at " + str(round(t_step[i_step[num]]*1e+3,3)) + " ms"
141         ax2[i][j].stairs(data_P_Qt[i_step[num]], Q_edges_data[i_step[num]], fill=True)
142         ax2[i][j].set_title(str_id)
143         ax2[i][j].axvline(x=0, color='black', linestyle='--',alpha=0.4, linewidth=1.0)
144         ax2[i][j].xaxis.set_major_formatter(FuncFormatter(lambda val, pos:
145             ↪ f"${val:.1f}$"))
146         if not (i == 0 and j == 0):
147             ax2[i][j].set_xlim(-10, 10)
148             ax2[i][j].set_ylim(0, 1.3)
149
150 fig2.supxlabel('${Q_t}$')
151 fig2.supylabel('P(${Q_t}$)')
152 plt.savefig(path+"2.png",dpi=200)
153
154 fig3 = plt.figure(num='Mean of Dissipation Function versus Time')
155 ax3 = fig3.gca()
156
157 data_mean_Q = []
158 for i in range(num_step):
159     data_Q_step = []
160     for Q in data_Q:
161         data_Q_step.append(Q[i])
162
163     freq, bins = np.histogram(data_Q_step,bins=bins,density=False)
164
165     Δx = np.diff(bins)
166     area = sum(freq*Δx)
167     P_Qt = freq/area
168
169     Qt = bins[:-1]
170     mean_Q = sum(P_Qt*Qt*Δx)
171
172     data_mean_Q.append(mean_Q)
173
174 plt.plot(t_step, data_mean_Q,'o',label = 'Simulation')
175
176 limit_mean_Q_predicted=(k0-k1)*((1/k1)-(1/k0))
177 limit_mean_Q_predicted=(1/2)*limit_mean_Q_predicted
178 τ = k1/y
179
180 data_mean_Q_predicted = []
181 for t in t_step:
182     mean_Q_predicted = (limit_mean_Q_predicted.x*(1-np.exp((-2*t*τ.x)))+(limit_mean_Q_
183     ↪ _predicted.y*(1-np.exp((-2*t*τ.y))))

```

```

183     data_mean_Q_predicted.append(mean_Q_predicted)
184     plt.plot(t_step, data_mean_Q_predicted, '-', color='red', label='Prediction')
185
186     ax3.legend()
187     plt.xlabel('t(s)')
188     plt.ylabel('<math>\langle Q_t \rangle</math>')
189     plt.savefig(path+"3.png",dpi=200)
190
191     i_step_ln = np.linspace(1, 6, 6, dtype=int)
192     size_i_step_ln = len(i_step_ln)
193     i_step_ln_custom = np.linspace(7, 50, 3, dtype=int)
194     i_step_ln = np.concatenate([i_step_ln,i_step_ln_custom])
195     size_i_step_ln += len(i_step_ln_custom)
196
197     fig4, ax4 = plt.subplots(num='Transient Fluctuation Theorem',nrows=3, ncols=3,
    ↪     figsize=(12, 12))
198
199     data_mean_ln = []
200     data_ln_data = []
201
202     data_mean_ln_predicted = []
203     data_ln_predicted = []
204
205     for k in range(size_i_step_ln):
206         data_Q_id_ld = copy.deepcopy(data_Q_list[i_step_ln[k]])
207
208         max_value = np.ceil(max(data_Q_id_ld))
209
210         freq_ln, edges_ln = np.histogram(data_Q_id_ld,bins=bins,density=False)
211
212         Δx_ln = np.diff(edges_ln)
213         area_ln = sum(freq_ln*Δx_ln)
214         P_Qi_ln = freq_ln/area_ln
215
216         mean = []
217         for i in range(len(edges_ln)-1):
218             mean.append((edges_ln[i]+edges_ln[i+1])/2)
219
220         ln_data = []
221
222         mean_ln = []
223         size = len(mean)-1
224         for i in range(len(mean)):
225             ln = P_Qi_ln[i]/P_Qi_ln[size-i]
226             ln_data.append(ln)
227             mean_ln.append(mean[i])
228
229         #delete NaN and inf
230         i = 0
231         while i < len(ln_data):
232             if math.isnan(ln_data[i]) or math.isinf(ln_data[i]):
233                 ln_data.pop(i)
234                 mean_ln.pop(i)
235                 i = i - 1
236             i = i + 1
237
238         data_mean_ln.append(mean_ln)
239         data_ln_data.append(ln_data)
240

```

```

241     ln_predicted = []
242     mean_ln_predicted = []
243     min_mean = np.ceil(min(mean_ln))
244     max_mean = np.ceil(max(mean_ln))
245     mean_linspace = np.linspace(min_mean, max_mean, 20)
246     for A in mean_linspace:
247         ln = np.exp(A)
248         ln_predicted.append(ln)
249         mean_ln_predicted.append(A)
250
251     data_mean_ln_predicted.append(mean_ln_predicted)
252     data_ln_predicted.append(ln_predicted)
253
254     for i in range(3):
255         for j in range(3):
256             num = j+i*3
257             str_id = "${Q_t}$ at " + str(round(t_step[i_step_ln[num]]*1e+3,3)) + " ms"
258             ax4[i][j].plot(data_mean_ln[num],data_ln_data[num],'o',label = 'Simulation')
259             ax4[i][j].plot(data_mean_ln_predicted[num],data_ln_predicted[num],'-',color=
↪ 'red', label='Prediction',zorder=0)
260             ax4[i][j].set_title(str_id)
261
262     fig4.supxlabel('${Q_t}$')
263     fig4.supylabel('P(${Q_t}$=A)/P(${Q_t}$=-A)')
264
265     plt.savefig(path+"4.png",dpi=200)
266
267     fig5, ax5 = plt.subplots(num='Logarithm of Transient Fluctuation Theorem',nrows=3,
↪ ncols=3, figsize=(12, 12))
268
269     data_mean_ln = []
270     data_ln_data = []
271
272     data_mean_ln_predicted = []
273     data_ln_predicted = []
274
275     for k in range(size_i_step_ln):
276         data_Q_id_ld = copy.deepcopy(data_Q_list[i_step_ln[k]])
277
278         max_value = np.ceil(max(data_Q_id_ld))
279
280         freq_ln, edges_ln = np.histogram(data_Q_id_ld,bins=bins,density=False)
281
282         Δx_ln = np.diff(edges_ln)
283         area_ln = sum(freq_ln*Δx_ln)
284         P_Qi_ln = freq_ln/area_ln
285
286         mean = []
287         for i in range(len(edges_ln)-1):
288             mean.append((edges_ln[i]+edges_ln[i+1])/2)
289
290         ln_data = []
291
292         mean_ln = []
293         size = len(mean)-1
294         for i in range(len(mean)):
295             if mean[i] >= 0:
296                 ln = np.log(P_Qi_ln[i]/P_Qi_ln[size-i])

```

```

297         ln_data.append(ln)
298         mean_ln.append(mean[i])
299
300     #delete NaN and inf
301     i = 0
302     while i < len(ln_data):
303         if math.isnan(ln_data[i]) or math.isinf(ln_data[i]):
304             ln_data.pop(i)
305             mean_ln.pop(i)
306             i = i - 1
307         i = i + 1
308
309     data_mean_ln.append(mean_ln)
310     data_ln_data.append(ln_data)
311
312     ln_predicted = []
313     mean_ln_predicted = []
314
315     if not mean_ln:
316         mean_ln.append(5)
317
318     max_mean = np.ceil(max(mean_ln))
319     mean_linspace = np.linspace(0, max_mean, 5)
320     for A in mean_linspace:
321         ln = np.log(np.exp(A))
322         ln_predicted.append(ln)
323         mean_ln_predicted.append(A)
324
325     data_mean_ln_predicted.append(mean_ln_predicted)
326     data_ln_predicted.append(ln_predicted)
327
328     for i in range(3):
329         for j in range(3):
330             num = j+i*3
331             str_id = "${Q_t}$ at " + str(round(t_step[i_step_ln[num]]*1e+3,3)) + " ms"
332             ax5[i][j].plot(data_mean_ln[num],data_ln_data[num],'o',label = 'Simulation')
333             ax5[i][j].plot(data_mean_ln_predicted[num],data_ln_predicted[num],'-',color=
↵ 'red', label='Prediction',zorder=0)
334             ax5[i][j].set_title(str_id)
335
336     fig5.supxlabel('${Q_t}$')
337     fig5.supylabel('ln[P(${Q_t}$=A)/P(${Q_t}$=-A)]')
338
339     plt.savefig(path+"5.png",dpi=200)
340
341     plt.show()

```

ประวัติผู้เขียน

ชื่อ-นามสกุล : นาย ปฐมพร เนียมรักษา
วัน เดือน ปีเกิด : 30 มีนาคม พ.ศ. 2541
สถานที่เกิด : รพ.สมเด็จพระปิ่นเกล้า กรมแพทย์ทหารเรือ
จังหวัดกรุงเทพมหานคร
วุฒิการศึกษา : สำเร็จการศึกษาระดับมัธยมศึกษาตอนปลาย
จาก ศรช.วัดบางปะกอก กศน.เขตราชวัตรบูรณะ
จังหวัดกรุงเทพมหานคร ปีการศึกษา 2559
สำเร็จปริญญาตรีวิทยาศาสตร์บัณฑิต (ฟิสิกส์)
จากมหาวิทยาลัยรามคำแหง ปีการศึกษา 2568